СОДЕРЖАНИЕ

СПИСОК ТЕРМИНОВ ... 3
ВВЕДЕНИЕ ... 8
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ЦИФРОВОЙ ДОСТУПНОСТИ ВЕБ-ИНТЕРФЕЙСОВ 12
1.1 Социальная и технологическая значимость цифровой доступности ... 12
1.2 Международные стандарты доступности веб-контента ... 13
1.3 Требования к доступности веб-ресурсов в Российской Федерации ... 15
1.4 Смысловая разметка веб-страниц (HTML, ARIA) и её роль в доступности интерфейсных компонентов ... 17
ГЛАВА 2. МЕТОДЫ И ИНСТРУМЕНТЫ АУДИТА ДОСТУПНОСТИ ВЕБ-СТРАНИЦ ... 19
2.1 Автоматизированные инструменты аудита доступности ... 19
2.1.1 Основные инструменты анализа доступности ... 19
2.1.2 Сравнение axe-core, WAVE, Lighthouse и Pa11y ... 19
2.1.3 Ограничения автоматизированного тестирования доступности ... 21
2.2 Ручной аудит и экспертная интерпретация нарушений ... 23
2.3 Компонентный подход к оценке пользовательской значимости ошибок ... 25
2.4 Практические проблемы массового аудита реальных веб-сайтов ... 26
2.5 Метрики доступности и структурной сложности веб-страниц ... 30
ГЛАВА 3. РАЗРАБОТКА И АПРОБАЦИЯ КОМПОНЕНТНО-ИНСТРУМЕНТАЛЬНОГО ПОДХОДА К АУДИТУ ДОСТУПНОСТИ ... 33
3.1 Исходные данные и структура набора веб-страниц ... 33
3.2 Программный конвейер сбора целевых показателей ... 36
3.2.1 Проверка функционирования с помощью библиотеки Playwright ... 37
3.2.2 Ожидание стабилизации страницы и обработка динамического контента ... 37
3.2.3 Сохранение страницы в формате MHTML с использованием Playwright ... 38
3.2.4 Извлечение HTML из MHTML ... 40
3.2.5. Сбор ключевых метрик: DOM, AXE, WAVE ... 41
3.3 Формирование аналитических таблиц и метрик исследования ... 47
3.4 Ручная компонентная переоценка выявленных нарушений ... 48
3.5 Статистический анализ результатов исследования ... 50
3.5.1 Логика проверки гипотез и используемые статистические методы ... 52
3.5.2 Проверка H1: структурная сложность обычных версий и версий для слабовидящих ... 54
3.5.3 Проверка H2: формальные показатели доступности обычных версий и версий для слабовидящих ... 55
3.5.4 Проверка H3: связь структурной сложности страницы с количеством нарушений ... 56
3.5.5 Проверка H4: автоматическая и экспертная оценка пользовательской значимости ошибок ... 57

3.5.6 Проверка H5: влияние изображений и визуальных элементов ... 58
3.5.7 Проверка H6: техническая нестабильность массового аудита ... 60
3.5.8 Сводный итог проверки гипотез ... 61
ГЛАВА 4. РЕКОМЕНДАТЕЛЬНАЯ МОДЕЛЬ АУДИТА И РАЗРАБОТКИ ДОСТУПНЫХ ВЕБ-ИНТЕРФЕЙСОВ ... 61
4.1 Контроль функционирования списка страниц ... 62
4.2 Сбор метрик страниц в автоматическом режиме ... 66
4.3 Компонентный режим ... 66
4.4 Преимущества компонентного режима ... 68
4.5 Рекомендации для разработки ... 70
ВЫВОДЫ ... 72
ОГРАНИЧЕНИЯ ИССЛЕДОВАНИЯ ... 74
НАПРАВЛЕНИЯ ДАЛЬНЕЙШИХ ИССЛЕДОВАНИЙ ... 76
ЗАКЛЮЧЕНИЕ ... 78
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ... 81

СПИСОК ТЕРМИНОВ

Автоматизированная оценка доступности (AIM-Score от WAVE) — интегральный количественный показатель доступности веб-страницы, рассчитываемый инструментом WAVE на основе выявленных нарушений стандартов WCAG; применяется для сравнительного анализа качества доступности пользовательских интерфейсов.

Алгоритм хеширования MD5 (Message Digest Algorithm 5) — криптографический алгоритм хеширования, генерирующий 128-битное хеш-значение (дайджест) для входных данных произвольной длины; в работе используется для идентификации файлов и верификации целостности данных.

Блокирующий уровень пользовательской значимости нарушения («ruin») — категория ручной оценки критичности нарушений доступности, обозначающая ошибку, полностью блокирующую выполнение целевого пользовательского сценария и делающую невозможным достижение поставленной пользователем цели

Версия для слабовидящих — адаптированная версия веб-страницы с модифицированным визуальным представлением интерфейса (увеличенный шрифт, контрастные цветовые схемы, упрощённая навигация), предназначенная для пользователей с нарушениями зрения; в рамках исследования соответствует метрикам с обозначением «low_vision».

Встроенный фрейм (inline frame, iframe) — HTML-элемент <iframe>, позволяющий встраивать содержимое отдельной веб-страницы или документа в текущую веб-страницу, обеспечивая независимую загрузку и отображение внешнего контента.

Доступные насыщенные интернет-приложения (Accessible Rich Internet Applications, ARIA) — набор атрибутов языка разметки, расширяющих семантику HTML для уточнения ролей, состояний и свойств интерактивных элементов интерфейса; предназначен для корректной интерпретации контента вспомогательными технологиями (например, программами экранного доступа).

Инструмент автоматизированного анализа доступности «Pa11y» — программный инструмент с открытым исходным кодом, реализующий автоматизированную проверку веб-страниц на соответствие стандартам доступности (в т.ч. WCAG) и формирующий структурированные отчёты о выявленных нарушениях.

Инструмент браузерной автоматизации «Playwright» — библиотека Python для автоматизированного управления веб-браузерами (Chromium, Chrome, Firefox, WebKit), используемая в работе для загрузки страниц, ожидания стабилизации динамического контента и сохранения состояния интерфейса на момент проведения аудита доступности.

Инструмент комплексного аудита веб-страниц «Lighthouse» — интегрированный в браузер Google Chrome инструмент анализа качества веб-страниц, оценивающий

производительность, доступность, SEO, прогрессивные веб-приложения и лучшие практики разработки на основе стандартизированных метрик.

Инструмент оценки веб-доступности «WAVE (Web Accessibility Evaluation Tool)» — веб-сервис для автоматизированной оценки доступности веб-страниц, визуализирующий проблемы доступности непосредственно на странице и формирующий детализированные отчёты с классификацией нарушений по уровням серьёзности.

Интерфейс — совокупность средств и правил взаимодействия между пользователем и программной системой, обеспечивающих управление функциями программы.

Инъекция скрипта — метод внедрения пользовательского JavaScript-кода в структуру веб-страницы на этапе её загрузки или выполнения, применяемый для модификации поведения интерфейса, автоматизации действий или сбора данных (в т.ч. в рамках аудита доступности).

Каскадные таблицы стилей (Cascading Style Sheets, CSS) — формальный язык описания внешнего вида веб-страниц, определяющий параметры отображения HTML-элементов (шрифты, цвета, отступы, позиционирование) и обеспечивающий разделение структуры и представления контента.

Классы каскадных таблиц стилей (CSS-классы) — именованные атрибуты HTML-элементов, используемые для группировки элементов интерфейса по функциональным или визуальным признакам и применения к ним унифицированных стилевых правил, определённых в таблицах стилей CSS.

Консорциум Всемирной паутины (World Wide Web Consortium, «W3C») — международная организация, координирующая разработку открытых стандартов веб-технологий, включая рекомендации по цифровой доступности (WCAG), семантической разметке (HTML, ARIA) и другим аспектам веб-разработки.

Консоль браузера (Browser Console) — встроенный в веб-браузер инструмент разработчика для выполнения JavaScript-кода, отладки скриптов, анализа ошибок выполнения и взаимодействия с объектной моделью документа (DOM) в реальном времени.

Критический уровень нарушения («critical») — высший уровень формальной тяжести нарушения доступности, обозначающий ошибку, оказывающую наиболее существенное негативное влияние на возможность использования интерфейса целевой аудиторией (в т.ч. пользователями с ограниченными возможностями здоровья).

Куки-файл (cookie) — небольшой текстовый фрагмент данных, сохраняемый веб-сайтом в браузере пользователя для хранения информации о сессии, настройках, предпочтениях или идентификации пользователя при повторных посещениях ресурса.

Модальное окно — элемент пользовательского интерфейса в виде всплывающего диалогового окна, блокирующего взаимодействие с основным содержимым страницы до выполнения требуемого действия (подтверждения, ввода данных и т. п.).

Наложенные средства доступности (accessibility overlays) — программные решения (скрипты, плагины), динамически добавляющие функции доступности (контрастность, навигация с клавиатуры и др.) поверх существующей веб-страницы без внесения изменений в её исходную разметку и код.

Незначительный уровень нарушения («minor») — низший уровень формальной тяжести нарушения доступности, выявляемый автоматизированными инструментами и обозначающий ошибку с минимальным предполагаемым влиянием на пользовательский опыт и доступность интерфейса.

Объектная модель документа (Document Object Model, DOM) — иерархическое дерево объектов, представляющее структуру и содержимое веб-страницы в виде узлов (элементов, атрибутов, текста); используется браузерами и программными инструментами для динамического доступа и изменения элементов интерфейса.

Обычная/стандартная версия веб-страницы — стандартная версия веб-страницы без специализированных режимов отображения (увеличенного шрифта, высокой контрастности и т. п.), предназначенная для пользователей без нарушений зрения; в исследовании соответствует метрикам с обозначением «normal».

Пользовательский интерфейс (User Interface, UI) — совокупность элементов взаимодействия между пользователем и программной системой, включающая кнопки, формы, ссылки, меню и другие элементы управления.

Пользовательский профиль браузера — набор пользовательских настроек и данных браузера, включающий установленные расширения, историю посещений, куки-файлы и другие параметры конфигурации.

Программа экранного доступа (screen reader) — специализированное программное обеспечение, преобразующее текстовую и графическую информацию, отображаемую на экране, в альтернативные формы вывода (синтезированную речь, тактильный вывод на брайлевские дисплеи) для пользователей с нарушениями зрения.

Программа экранного доступа «JAWS» (Job Access With Speech) — коммерческая программа экранного доступа для операционных систем семейства Microsoft Windows.

Программа экранного доступа «NVDA» (NonVisual Desktop Access) — бесплатная программа экранного доступа с открытым исходным кодом для Microsoft Windows, предоставляющая пользователям с нарушениями зрения возможность взаимодействия с графическим интерфейсом посредством речи и брайлевских дисплеев.

Программа экранного доступа «VoiceOver» — встроенная программа экранного доступа в экосистеме Apple (macOS, iOS), обеспечивающая голосовое сопровождение интерфейса и навигацию с помощью жестов и клавиатурных команд для пользователей с нарушениями зрения.

Программный движок автоматизированной проверки доступности (axe-core) — библиотека с открытым исходным кодом для автоматизированного тестирования веб-страниц на соответствие стандартам доступности (WCAG); анализирует структуру DOM и выявляет нарушения, связанные с семантикой, клавиатурной навигацией, контрастом и т.д.

Протокол инструментов разработчика «Chrome» (Chrome DevTools Protocol, «CDP») — программный интерфейс (API) для взаимодействия с браузером на базе Chromium, позволяющий управлять браузером, получать данные о состоянии страницы, выполнять автоматизированные действия и отлаживать код через инструменты разработчика.

Расширение браузера — дополнительный программный модуль, устанавливаемый в веб-браузер для расширения его функциональных возможностей (добавления новых инструментов, модификации интерфейса, автоматизации задач).

Рекомендации по доступности веб-контента «WCAG (Web Content Accessibility Guidelines)» — международный стандарт рекомендаций по обеспечению доступности веб-контента, разработанный W3C в рамках Инициативы веб-доступности (WAI); содержит критерии и методики создания интерфейсов, доступных для пользователей с различными ограничениями.

Серьёзный уровень нарушения («serious») — уровень формальной тяжести нарушения доступности, обозначающий ошибку, оказывающую значительное негативное влияние на пользовательский опыт и затрудняющую выполнение ключевых сценариев взаимодействия с интерфейсом.

Спецификация WAI-ARIA (Web Accessibility Initiative — Accessible Rich Internet Applications / Инициатива веб-доступности — доступные насыщенные интернет-приложения) — стандартизированный набор спецификаций Консорциума Всемирной паутины (World Wide Web Consortium, W3C), предназначенный для повышения уровня доступности динамического веб-контента и интерактивных пользовательских интерфейсов веб-приложений

Удобство использования — качественная характеристика пользовательского интерфейса, отражающая степень лёгкости, интуитивности и эффективности взаимодействия пользователя с веб-сайтом при выполнении целевых задач.

Умеренный уровень нарушения («moderate») — уровень формальной тяжести нарушения доступности, обозначающий ошибку со средним предполагаемым влиянием на пользовательский опыт, требующую устранения для повышения общей доступности интерфейса.

Формат сохранения веб-страницы MHTML (MIME HTML) — формат архивного сохранения веб-страницы вместе со связанными ресурсами, позволяющий зафиксировать состояние страницы на момент анализа.

Элементы языка гипертекстовой разметки (HTML-элементы) — структурные компоненты веб-страницы, представляющие различные типы содержимого и интерфейса, например заголовки, ссылки, изображения, формы и списки.

Язык программирования JavaScript — язык программирования, применяемый для реализации динамического поведения веб-страниц и выполнения сценариев в браузере.

Язык программирования Python — язык программирования, применяемый в работе для реализации скриптов сбора, обработки и статистического анализа данных.

Язык гипертекстовой разметки HTML (HyperText Markup Language) — язык разметки, определяющий структуру веб-страницы и её элементы.

Chromium — открытый проект веб-браузера и программного движка, лежащий в основе ряда современных браузеров (в т.ч. Google Chrome, Microsoft Edge, Opera); обеспечивает реализацию стандартов веб-технологий (HTML, CSS, JavaScript, Web APIs) и предоставляет программные интерфейсы (в т.ч. Chrome DevTools Protocol) для управления браузером, анализа страниц и автоматизации взаимодействия с веб-контентом; в работе используется как платформа для проведения автоматизированного аудита доступности веб-страниц.

Playwright Inspector — интерактивный инструмент отладки и визуализации в составе фреймворка Playwright, предоставляющий графический интерфейс для пошагового выполнения автоматизированных тестов, инспекции элементов веб-страницы, анализа сетевых запросов и отслеживания состояния браузера в режиме реального времени; используется для отладки сценариев браузерной автоматизации и верификации корректности взаимодействия с веб-интерфейсами.

ВВЕДЕНИЕ

Актуальность темы исследования обусловлена тем, что значительная часть государственных услуг и информации предоставляется через интернет-платформы. Поэтому обеспечение доступности веб-ресурсов приобретает особую значимость для пользователей с различными ограничениями здоровья, включая людей с нарушениями зрения [1].

Проблема цифровой доступности не ограничивается только пользователями, имеющими официально установленную инвалидность. Различные ограничения взаимодействия с цифровыми интерфейсами могут быть постоянными, временными или ситуативными: временное нарушение зрения, использование мобильного устройства, низкая освещённость, возрастные особенности или иных факторов [2]. Следовательно, доступность веб-интерфейсов является не узкоспециализированной, а общей характеристикой качества цифровой среды.

Особую роль в данном контексте играют государственные веб-сайты, поскольку они обеспечивают доступ граждан к официальной информации, электронным услугам, нормативным материалам и каналам взаимодействия с органами власти. Одним из распространённых способов адаптации государственных сайтов является создание специальных версий страниц для пользователей с нарушениями зрения. Однако наличие такой версии само по себе не гарантирует фактического повышения доступности [3, 4].

В связи с этим возникает необходимость не только сравнивать обычные и адаптированные версии веб-страниц, но и анализировать, насколько применяемые методы оценки доступности способны отражать реальное состояние интерфейса. В современных исследованиях и практическом аудите широко используются автоматизированные инструменты, такие как axe-core [5] (официальная библиотека для автоматизированного тестирования доступности от Deque Systems) и WAVE [6] (онлайн-инструмент WebAIM для визуализации проблем доступности). Они позволяют выявлять нарушения стандартов WCAG, анализировать структуру страницы и получать сопоставимые количественные показатели для большого набора веб-ресурсов [7,8].

Вместе с тем результаты автоматизированного аудита не могут быть напрямую приравнены к фактической пользовательской доступности. Такие инструменты фиксируют нарушения на уровне формальных правил проверки, но не всегда позволяют определить, к какому интерфейсному компоненту относится ошибка и насколько существенно она влияет на выполнение пользовательского сценария. Дополнительным ограничением аудита является техническая нестабильность реальных веб-сайтов: ошибки загрузки, блокировки автоматизированных запросов, ограничения выполнения JavaScript-кода, динамическое изменение контента и проблемы сохранения состояния страницы. Поэтому контроль доступности веб-ресурса рассматривается одним из независимых сценариев доступности веб-ресурса для пользователя и исследователей.

Для решения поставленных проблем в работе предлагается компонентно-инструментальный подход к аудиту доступности государственных веб-сайтов. Под компонентным аспектом понимается сопоставление выявленных нарушений с конкретными элементами интерфейса, анализ их разметки и семантики, а также ручная переоценка фактической значимости связанных ошибок. Под инструментальным аспектом понимается разработка контролируемого процесса сбора данных, позволяющего фиксировать состояние страницы, получать связанный набор данных о состоянии страницы в момент анализа: DOM-структуру, результаты axe-core, показатели WAVE , сведения о видимости элементов, ошибки навигации и признаки некорректного завершения аудита.

В рамках данной работы обычные версии и версии для слабовидящих государственных веб-страниц рассматриваются как эмпирический материал для апробации предложенного подхода. Их сравнение позволяет оценить, приводит ли специальный режим отображения к улучшению формальных метрик доступности, изменению структуры страницы и снижению количества выявленных нарушений. Одновременно ручная компонентная переоценка позволяет проверить, насколько автоматические показатели соответствуют фактической значимости ошибок для пользовательского взаимодействия.

Цель исследования — разработать и апробировать компонентно-инструментальный подход к аудиту доступности государственных веб-сайтов, объединяющий автоматизированный сбор метрик доступности, контроль технического состояния страниц, сравнение базовой и адаптированной для слабовидящих версий и ручную компонентную переоценку пользовательской значимости выявленных нарушений.

Объектом исследования являются веб-интерфейсы государственных сайтов, представленные в обычных версиях и версиях для пользователей с нарушениями зрения.

Предметом исследования является процесс автоматизированного и экспертного аудита доступности, в т.ч. компонентно-инструментальный анализ доступности веб-страниц, включающий анализ:

- структуры объектной модели документа, DOM-структуру;
- семантики гипертекстовой разметки, HTML;
- семантики специальных атрибутов для обеспечения доступности компонентов (ARIA, Accessible Rich Internet Applications);
- результатов автоматизированных проверок с помощью инструментов анализа доступности axe-core, WAVE;
- технических характеристик состояния страниц при сборе данных;
- ручной переоценки пользовательской значимости выявленных нарушений.

Для достижения поставленной цели были сформулированы следующие задачи исследования:

1) Рассмотреть теоретические основы цифровой доступности, международные стандарты обеспечения цифровой доступности веб-контента, WCAG, роль смысловой разметки гипертекстового документа, HTML, и специальных атрибутов для поддержки вспомогательных технологий, ARIA, а также особенности взаимодействия пользователей со вспомогательными технологиями.

2) Проанализировать существующие подходы к автоматизированному и ручному аудиту доступности веб-страниц, а также выявить ограничения автоматических инструментов при интерпретации пользовательской значимости ошибок.

3) Сформировать набор данных государственных веб-страниц, включающий базовые версии страниц и версии для пользователей с нарушениями зрения (адаптированные версии).

4) Разработать и применить программный комплекс для массовой проверки доступности, обеспечивающий:

сбор информации о внутренней структуре документа;

сведений о видимости элементов на экране;

результатов проверок сайтов с помощью автоматизированных инструментов анализа ;

выявления ошибок навигации на веб-странице;

признаков некорректного состояния веб-страницы.

5) Определить систему количественных метрик для анализа веб-страниц, включающую:

характеристики внутренней структуры документа;

количество и степень серьёзности нарушений, выявленных методом автоматизированного анализа исходного кода веб-страниц;

данные проверок интерфейса с применением средств контекстной визуализации данных;

долю ошибок относительно сложности страницы;

показатели, рассчитанные с учётом и без учёта графических элементов.

6) Выполнить сравнительный анализ базовой и адаптированной для слабовидящих версий страниц с точки зрения структурной сложности интерфейса, количества выявленных нарушений и интегральных показателей доступности.

7) Провести ручную компонентную переоценку части выявленных ошибок: сопоставить автоматические нарушения с интерфейсными компонентами и пользовательской значимостью этих нарушений.

8) Сравнить автоматическую и экспертную оценку критичности нарушений доступности и определить случаи, когда формальная тяжесть ошибки, по данным программ, не совпадает с её фактическим влиянием на пользовательский опыт.

9) Проанализировать технические ограничения массового аудита доступности.

10) Разработать рекомендации по аудиту и разработке доступных веб-интерфейсов на основе результатов автоматизированного анализа, ручной компонентной переоценки и контроля технического состояния страниц.

Структура работы соответствует логике поставленной цели и включает теоретическую, методическую и эмпирическую части. В первой главе рассматриваются теоретические основы цифровой доступности, международные и российские требования к доступности веб-ресурсов, роль международных стандартов обеспечения доступности веб-контента, значение разметки гипертекстового документа и специальных атрибутов для поддержки вспомогательных технологий и технологий в обеспечении доступного взаимодействия пользователя с веб-интерфейсом.

Во второй главе анализируются существующие подходы к автоматизированному и ручному аудиту доступности. Особое внимание уделяется возможностям и ограничениям инструментов axe-core и WAVE, проблеме интерпретации автоматических ошибок, а также различию между формальной критичностью нарушения и его фактической пользовательской значимостью.

В третьей главе описывается разработанный компонентно-инструментальный подход к аудиту доступности государственных веб-сайтов. В рамках данной главы рассматриваются формирование корпуса веб-страниц, сбор внутренней структуры документа, получение результатов инструментов анализа доступности axe-core и WAVE, контроль технического состояния страниц, построение таблиц для статистического анализа и процедура ручной компонентной переоценки выявленных нарушений.

Эмпирическая часть работы включает сравнение базовой и адаптированной для слабовидящих государственных веб-страниц по структурным характеристикам интерфейса, автоматическим метрикам доступности и интегральным показателем доступности (WAVE/AIM). Дополнительно анализируется влияние изображений и визуальных элементов на количество формальных ошибок, связь между сложностью внутренней структуры документа и числом нарушений, а также расхождения между автоматической и экспертной оценкой критичности ошибок.

В заключительной части работы обсуждаются полученные результаты, ограничения автоматизированного аудита, технические проблемы массового сбора данных с реальных веб-сайтов и условия, при которых результаты автоматической проверки требуют ручной верификации. На основе проведённого анализа формулируются рекомендации по применению автоматических инструментов аудита совместно с компонентной интерпретацией ошибок и контролем состояния анализируемой страницы.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ЦИФРОВОЙ ДОСТУПНОСТИ ВЕБ-ИНТЕРФЕЙСОВ

1.1 Социальная и технологическая значимость цифровой доступности

Развитие цифровых технологий и широкое распространение интернет-сервисов привели к тому, что значительная часть государственных, образовательных и коммерческих услуг предоставляется через веб-интерфейсы. В условиях цифровизации общества обеспечение равного доступа к информации становится важной социальной и технологической задачей. Особенно актуальной данная проблема является для людей с инвалидностью, в частности для пользователей с нарушениями зрения.

Цифровая доступность предполагает разработку веб-ресурсов таким образом, чтобы они могли использоваться максимально широким кругом пользователей независимо от их физических возможностей и используемых вспомогательных технологий. К таким технологиям относятся экранные дикторы, программы увеличения изображения, альтернативные устройства ввода и другие инструменты, обеспечивающие взаимодействие пользователей с веб-контентом [8].

Недостаточный уровень доступности веб-ресурсов приводит к формированию цифрового неравенства [9]. Пользователи с ограниченными возможностями здоровья сталкиваются с многочисленными барьерами при использовании интернет-сервисов, что ограничивает их доступ к образовательным ресурсам, государственным услугам и цифровой инфраструктуре. В ряде исследований отмечается, что значительная часть веб-сайтов образовательных организаций не соответствует установленным требованиям доступности и содержит множество препятствий для пользователей с нарушениями зрения [10, 11].

Следует также отметить различие между понятиями доступности и удобства использования. Веб-сайт может быть формально доступным с точки зрения технических требований, но при этом оставаться сложным для практического использования пользователями. Исследования показывают, что уровень доступности не всегда напрямую коррелирует с уровнем юзабилити интерфейса [12].

Современные подходы к оценке доступности веб-ресурсов предполагают использование количественных метрик и автоматизированных инструментов анализа. Такие методы позволяют выявлять типовые проблемы веб-интерфейсов и проводить сравнительный анализ различных сайтов [7]. Развитие инструментов автоматизированного тестирования делает возможным проведение массового анализа доступности веб-ресурсов и выявление наиболее распространённых нарушений стандартов.

В последние годы активно обсуждается проблема использования «accessibility overlays» — специальных скриптов и плагинов, которые добавляют на веб-страницу дополнительные функции доступности. Популярность подобных решений обусловлена их сравнительно низкой стоимостью

внедрения и возможностью продемонстрировать формальные меры по адаптации интерфейса без глубокой переработки структуры сайта. Исследования показывают, что подобные инструменты не устраняют фундаментальные проблемы архитектуры веб-интерфейсов и часто не обеспечивают соответствие стандартам доступности [3]. Они не исправляют некорректную семантическую разметку, отсутствие связей между элементами формы, ошибки структуры заголовков, нарушения клавиатурной навигации и иные проблемы, заложенные в исходном коде интерфейса. В результате пользователь получает дополнительный слой управления, но не гарантированно получает доступ к содержанию и функциям сайта.

Следовательно, overlays не могут рассматриваться как полноценная альтернатива систематическому обеспечению доступности на уровне проектирования, разработки и тестирования веб-ресурса. Их применение может быть допустимо лишь как вспомогательная мера, но не как замена соблюдению требований WCAG и проведению комплексного аудита.

Таким образом, обеспечение цифровой доступности веб-ресурсов является важным направлением развития информационных технологий. Разработка и внедрение стандартов доступности позволяет формировать единые требования к проектированию веб-интерфейсов и обеспечивает возможность системной оценки уровня доступности интернет-ресурсов.

1.2 Международные стандарты доступности веб-контента

Наиболее значимую роль в формировании международных стандартов цифровой доступности играет Консорциум Всемирной паутины (World Wide Web Consortium, W3C). В рамках Инициативы по обеспечению веб-доступности (Web Accessibility Initiative, WAI) были разработаны Рекомендации по обеспечению доступности веб-контента (Web Content Accessibility Guidelines, WCAG), которые используются как базовый международный стандарт обеспечения доступности веб-контента [13].

Развитие рекомендаций WCAG происходило поэтапно и отражало эволюцию веб-технологий. Первая версия стандарта, WCAG 1.0, была ориентирована преимущественно на статические веб-страницы. Переход к WCAG 2.0 сопровождался формированием технологически нейтральной модели критериев соответствия, что позволило применять рекомендации к более широкому кругу интерфейсов. Последующие редакции, WCAG 2.1 и WCAG 2.2, уточнили требования с учётом мобильных устройств, сенсорного ввода, масштабирования, устойчивости интерфейса и особенностей взаимодействия пользователей с когнитивными и моторными ограничениями.

Структурной основой WCAG выступают четыре базовых принципа: воспринимаемость, управляемость, понятность и надёжность. Принцип воспринимаемости означает, что информация и элементы интерфейса должны быть представлены в формах, доступных для восприятия

пользователем, например за счёт текстовых альтернатив и достаточного контраста. Принцип управляемости предполагает возможность взаимодействия с интерфейсом различными способами, включая клавиатурную навигацию. Понятность связана с предсказуемостью поведения интерфейса и ясностью представления информации. Надёжность означает совместимость контента с браузерами и вспомогательными технологиями, включая программы экранного доступа.

Практическая значимость стандартов WCAG определяется не только их нормативным статусом, но и тем, что они описывают требования с учётом реальных сценариев взаимодействия пользователей со вспомогательными технологиями. Экранные дикторы, программы увеличения изображения, клавиатурная навигация, альтернативные устройства ввода и брайлевские дисплеи предъявляют к структуре документа и семантике интерфейса особые требования. Поэтому соблюдение WCAG следует рассматривать как условие корректного взаимодействия веб-ресурса с целым классом технических средств, а не исключительно как набор формальных критериев соответствия.

Стандарт WCAG определяет требования к веб-ресурсам, направленные на обеспечение их доступности для пользователей с различными видами инвалидности, включая нарушения зрения, слуха, моторики и когнитивные особенности [14]. Основой стандарта являются четыре фундаментальных принципа доступности: воспринимаемость, управляемость, понятность и надёжность. Каждый из этих принципов включает набор критериев соответствия, которые должны соблюдаться при разработке веб-контента.

Современной версией данного стандарта является WCAG 2.2, который учитывает особенности использования современных веб-интерфейсов, включая мобильные устройства и динамические веб-приложения [15]. В рамках стандарта предусмотрены три уровня соответствия: A, AA и AAA. Уровень AA считается базовым уровнем соответствия для большинства государственных и коммерческих веб-ресурсов.

В настоящее время ведётся разработка следующего поколения стандарта доступности веб-контента — WCAG 3.0 (рабочее название проекта — *«Silver»*), который предполагает изменение подходов к оценке доступности цифровых интерфейсов. В отличие от предыдущих версий стандарта, основанных на бинарной проверке выполнения отдельных критериев соответствия, новая версия ориентирована на более гибкую систему комплексной оценки. Предлагается использование количественных показателей доступности и агрегированных оценок, учитывающих степень выполнения требований и их влияние на пользователей. Кроме того, в рамках WCAG 3.0 предполагается комбинирование различных методов анализа, включая автоматическое тестирование, экспертную оценку и пользовательское тестирование, что должно обеспечить более точное отражение реального качества доступности современных веб-интерфейсов.[16]

Рекомендации WCAG также были формализованы в международном стандарте ISO/IEC 40500:2012, что закрепило их в качестве официального международного стандарта цифровой доступности [14]. Это позволило использовать данные требования в качестве нормативной базы для разработки веб-ресурсов в различных странах.

Для повышения доступности сложных интернет-интерфейсов используется технология доступных насыщенных веб-приложений (Accessible Rich Internet Applications, ARIA). Руководство по авторским практикам (ARIA Authoring Practices) [17] содержит рекомендации по использованию специальных атрибутов, позволяющих улучшить взаимодействие вспомогательных технологий с динамическими элементами веб-страниц.

Помимо нормативных стандартов, значительный вклад в развитие методов оценки доступности внесли научные исследования. Например, в работе M. Vigo, M. Arrue, G. Brajnik, R. Lomuscio и J. Abascal предложена система количественных метрик для измерения уровня доступности веб-страниц [18]. Эти метрики позволяют проводить сравнительный анализ веб-ресурсов, выявлять типовые проблемы интерфейсов.

Позднейшие исследования также рассматривали эффективность различных инструментов автоматизированной оценки доступности. Результаты показали, что разные инструменты могут обнаруживать различные типы нарушений стандартов, что подчёркивает необходимость комбинирования автоматических и экспертных методов анализа [8].

Таким образом, международные стандарты и рекомендации формируют основу для разработки доступных веб-ресурсов и создают единый принцип оценки доступности цифровых интерфейсов.

1.3 Требования к доступности веб-ресурсов в Российской Федерации

В Российской Федерации вопросы обеспечения доступности цифровых ресурсов регулируются рядом нормативных документов, направленных на обеспечение равного доступа к информации для людей с инвалидностью. Формирование требований к доступности цифровых сервисов связано с развитием государственной политики в области социальной интеграции и создания безбарьерной информационной среды.

Базовым нормативным актом, регулирующим права людей с инвалидностью в Российской Федерации, является Федеральный закон №181-ФЗ «О социальной защите инвалидов в Российской Федерации». Данный закон закрепляет принцип равного доступа людей с инвалидностью к информации, услугам и объектам социальной инфраструктуры. В контексте цифровой среды положения данного закона распространяются также на информационные системы и интернет-ресурсы, которые должны быть адаптированы для использования людьми с различными ограничениями здоровья.[19]

Развитие требований к цифровой доступности отражено и в системе национальных стандартов Российской Федерации. В частности, ГОСТ Р 52872-2019 «Интернет-ресурсы и другая информация, представленная в электронно-цифровой форме. Приложения для стационарных и мобильных устройств, иные пользовательские интерфейсы. Требования доступности для инвалидов по зрению» устанавливает требования к структуре и представлению информации в цифровых интерфейсах. Данный стандарт определяет рекомендации по обеспечению корректного отображения текстовой информации, использованию альтернативных описаний для графических элементов, обеспечению достаточного контраста элементов интерфейса и поддержке работы вспомогательных технологий.[20]

Кроме того, в нормативной базе Российской Федерации используются международные стандарты эргономики взаимодействия человека и компьютерных систем. К таким стандартам относится ГОСТ Р ИСО 9241-20 «Эргономика взаимодействия человек–система. Часть 20. Руководство по обеспечению доступности информационно-коммуникационных технологий». Данный стандарт определяет общие принципы проектирования интерфейсов с учётом потребностей пользователей с различными ограничениями здоровья. В частности, он содержит рекомендации по разработке интерфейсов, поддерживающих использование вспомогательных технологий, включая экранные дикторы, альтернативные устройства ввода и программные средства увеличения изображения.[21]

Дополнительные аспекты обеспечения доступности рассматриваются в стандарте ГОСТ Р ИСО 9999, который содержит классификацию технических средств реабилитации для людей с инвалидностью. Хотя данный стандарт непосредственно не регулирует структуру веб-интерфейсов, он определяет категории вспомогательных технологий, используемых пользователями с различными ограничениями здоровья. Учет особенностей таких технологий является важным фактором при разработке доступных веб-ресурсов и цифровых сервисов.[22]

Одним из ключевых нормативных актов является приказ Министерства цифрового развития, связи и массовых коммуникаций Российской Федерации №953 от 7 ноября 2023 года. Данный документ устанавливает порядок обеспечения условий доступности для инвалидов по зрению официальных сайтов государственных органов, органов местного самоуправления и подведомственных организаций.[23]

Согласно данному приказу, официальные сайты государственных организаций должны обеспечивать возможность использования специальных режимов отображения информации для пользователей с нарушениями зрения. Такие режимы предусматривают изменение цветовой схемы, увеличение размера текста, а также упрощение структуры веб-страниц.

Особое внимание в нормативных требованиях уделяется корректной структуре веб-страниц и использованию текстовых альтернатив для графических элементов. Эти требования направлены на обеспечение корректной работы вспомогательных технологий, таких как экранные дикторы.

Исследования доступности российских веб-ресурсов показывают, что несмотря на наличие нормативных требований, уровень доступности многих сайтов остаётся недостаточным. В частности, анализ сайтов российских университетов выявил значительное количество нарушений стандартов доступности, включая отсутствие альтернативных текстов для изображений, проблемы контрастности и некорректную структуру заголовков [10].

В научной литературе также рассматриваются вопросы формирования требований к интерфейсам образовательных сайтов с учётом стандартов доступности и удобства использования. Исследователи отмечают необходимость комплексного подхода к проектированию веб-интерфейсов, учитывающего как технические стандарты, так и реальные потребности пользователей с ограниченными возможностями здоровья [24, 25].

Таким образом, нормативная база Российской Федерации в области цифровой доступности постепенно развивается и формирует требования к разработке и поддержке доступных веб-ресурсов. Однако для обеспечения реального соответствия данным требованиям необходимы не только формальные механизмы адаптации интерфейсов, но и систематическая проверка доступности сайтов с использованием методов автоматического и экспертного аудита.

1.4 Смысловая разметка веб-страниц (HTML, ARIA) и её роль в доступности интерфейсных компонентов.

Доступность веб-интерфейса определяется не только визуальным представлением страницы, но и тем, насколько корректно её структура интерпретируется браузерами, вспомогательными технологиями и автоматизированными инструментами аудита. Пользователь, работающий с экранным диктором, клавиатурой или иным вспомогательным средством, взаимодействует не с визуальным макетом страницы напрямую, а с её семантической структурой. Поэтому качество HTML-разметки и корректность применения «ARIA»-атрибутов являются важными условиями фактической доступности веб-интерфейса.

Семантическая HTML-разметка позволяет явно обозначать назначение элементов страницы. Например, элементы <button>, <a>, <nav>, <main>, <form>, <label>, <header> и <footer> несут не только визуальную, но и функциональную информацию. Если интерактивный элемент реализован с помощью нейтрального контейнера, например или , без указания роли, имени и состояния, он может быть визуально понятен зрячему пользователю, но недоступен или некорректно представлен для пользователя экранного диктора.

Технология «WAI-ARIA» используется для уточнения ролей, состояний и свойств элементов интерфейса в тех случаях, когда стандартной HTML-семантики недостаточно. С помощью «ARIA»-атрибутов можно сообщать вспомогательным технологиям, что элемент является кнопкой, вкладкой, раскрывающимся списком, модальным окном, областью навигации или другим компонентом интерфейса. Атрибутирование не заменяет корректную HTML-разметку и при неправильном использовании может не улучшать, а ухудшать доступность: некорректно заданная роль, отсутствие доступного имени, неправильное состояние <aria-expanded> или использование <aria-hidden> для значимого содержимого могут привести к тому, что пользователь не сможет правильно интерпретировать или использовать компонент.

Особое значение имеет то, что пользовательский интерфейс состоит не из отдельных изолированных элементов, а из компонентов: навигационных меню, карточек, форм, кнопок, ссылок, списков, модальных окон, выпадающих элементов, блоков поиска и других функциональных частей страницы. Каждый компонент имеет определённую пользовательскую роль и включён в конкретный сценарий взаимодействия. Поэтому одно и то же формальное нарушение доступности может иметь разную фактическую значимость в зависимости от того, в каком компоненте оно находится.

Например, отсутствие доступного имени у кнопки отправки формы может полностью заблокировать выполнение пользовательского сценария. В то же время отсутствие альтернативного описания у декоративного изображения может быть зафиксировано автоматическим инструментом как ошибка, но иметь значительно меньшую практическую значимость, если изображение не несёт смысловой функции. Аналогично ошибка в основном навигационном меню может быть критичнее, чем ошибка в повторяющемся второстепенном блоке страницы.

Таким образом, анализ доступности только на уровне общего количества ошибок не позволяет достаточно точно оценить фактическое влияние нарушений на пользователя. Для более содержательной интерпретации необходимо учитывать, к какому интерфейсному компоненту относится ошибка, какую функцию выполняет данный компонент, является ли он частью основного пользовательского сценария и каким образом нарушение влияет на взаимодействие с ним.

В рамках данной работы компонентный подход используется для сопоставления автоматических ошибок доступности с конкретными элементами интерфейса и последующей ручной переоценки их пользовательской значимости. Такой подход позволяет дополнить формальные результаты автоматизированного аудита анализом HTML/ARIA-семантики и контекста использования компонента.

ГЛАВА 2. МЕТОДЫ И ИНСТРУМЕНТЫ АУДИТА ДОСТУПНОСТИ ВЕБ-СТРАНИЦ

2.1 Автоматизированные инструменты аудита доступности

2.1.1 Основные инструменты анализа доступности

Автоматические средства тестирования применяются для оперативного выявления типовых нарушений доступности и широко используются как в практике разработки, так и в массовых исследованиях веб-ресурсов. Их основное достоинство заключается в возможности анализировать большое число страниц по единым правилам и получать сопоставимые количественные показатели.

В рамках настоящего исследования основное внимание уделяется инструментам axe-core и WAVE. Библиотека axe-core представляет собой программный движок автоматической проверки, ориентированный на анализ DOM-структуры страницы и выявление нарушений, которые могут быть надёжно установлены на основе HTML-разметки, атрибутов доступности и свойств интерфейсных элементов [5]. Инструмент WAVE, в свою очередь, сочетает автоматическую проверку с визуальным представлением результатов, что делает его удобным для интерпретации ошибок на уровне интерфейса [6].

Дополнительно в профессиональной практике используются «Lighthouse» и «Pa11y». «Lighthouse» интегрирован в экосистему браузера «Chrome» и позволяет выполнять комплексную оценку качества страницы, включая производительность и доступность. «Pa11y» ориентирован на автоматизированные проверки в среде непрерывной интеграции и удобен в тех случаях, когда структура тестируемых страниц заранее определена и сценарии проверки могут быть формализованы.[26,27]

Использование различных инструментов анализа доступности позволяет получать более полное представление о состоянии веб-ресурсов, поскольку разные системы могут выявлять различные типы нарушений. Однако выбор инструментов для конкретного исследования определяется не только их функциональными возможностями, но и техническими условиями проведения анализа.

2.1.2 Сравнение axe-core, WAVE, Lighthouse и Pa11y

Автоматические инструменты анализа доступности различаются не только набором реализованных правил проверки, но и принципами интерпретации обнаруженных проблем.

Инструмент axe-core ориентирован на обнаружение нарушений, которые могут быть автоматически подтверждены на основе структуры HTML-документа и атрибутов доступности. Философия данного инструмента анализа лежит в нулевой толерантности к ложным срабатываниям [5], одновременно ограничивает число выявляемых проблем, поскольку многие аспекты доступности требуют содержательной интерпретации интерфейса.

Инструмент WAVE, напротив, в большей степени ориентирован на поддержку ручного аудита. Он визуально отображает элементы страницы и предупреждения о потенциальных проблемах доступности непосредственно в интерфейсе веб-страницы. Такой подход облегчает анализ структуры страницы человеком, однако может сопровождаться более высоким числом предупреждений, требующих дополнительной проверки.

Инструменты «Lighthouse» и «Pa11y» чаще используются в стандартизированных сценариях проверки веб-приложений. «Lighthouse» обеспечивает быстрый автоматический аудит страницы непосредственно в браузере, тогда как «Pa11y» применяется преимущественно в системах непрерывной интеграции для регулярного тестирования доступности разрабатываемых веб-интерфейсов.

Для наглядного сопоставления характеристик рассматриваемых инструментов результаты анализа сведены в таблицу 1.

Таблица 1 — Сравнение инструментов автоматического анализа доступности

Инструмент	Тип инструмента	Основной принцип работы	Преимущества	Ограничения
axe-core	программный движок проверки	анализ DOM и правил WCAG	высокая точность, открытый исходный код, нулевая толерантность к ложным срабатываниям	не выявляет проблемы, требующие контекстной оценки
WAVE	визуальный инструмент аудита	автоматический анализ, визуальная маркировка элементов	удобство интерпретации результатов	большое количество предупреждений
Lighthouse	встроенный инструмент браузера	комплексный аудит страницы	быстрая оценка производительности и доступности	ограниченная гибкость при анализе внешних сайтов

Pa11y	инструмент «CI/CD»	автоматические тесты по заданным сценариям	удобен для непрерывного тестирования	требует заранее определённых сценариев

Представленное сравнение показывает, что различные инструменты ориентированы на разные задачи анализа. axe-core обеспечивает строгую автоматическую проверку, тогда как WAVE облегчает интерпретацию результатов на уровне интерфейса. «Lighthouse» и «Pa11y» удобны для стандартизированных сценариев тестирования, однако их применение может быть ограничено в условиях анализа внешних веб-ресурсов.

2.1.3 Ограничения автоматизированного тестирования доступности

Несмотря на широкое распространение автоматических инструментов анализа доступности, их использование связано с рядом методологических и технических ограничений.

Прежде всего автоматические средства способны выявлять только те нарушения, которые могут быть формализованы и обнаружены на основе анализа структуры веб-документа. Многие аспекты доступности связаны с семантической интерпретацией контента и качеством пользовательского взаимодействия. Например, инструмент может определить наличие альтернативного текста у изображения, однако не способен автоматически оценить его содержательную корректность.

Дополнительные ограничения связаны с особенностями современных веб-приложений. Многие сайты используют динамически генерируемый контент, асинхронную загрузку элементов интерфейса и сложные клиентские сценарии взаимодействия. В таких условиях часть элементов страницы может появляться только после выполнения действий пользователя, что затрудняет их анализ автоматическими инструментами.

При проведении массового анализа внешних веб-ресурсов возникают также ограничения, связанные с сетевой доступностью страниц. Некоторые сайты могут блокировать автоматические запросы, ограничивать доступ по географическому признаку или использовать устаревшие протоколы шифрования. В результате автоматический запуск инструментов анализа может завершаться ошибкой соединения или блокировкой загрузки страницы.

Отдельной проблемой является различие в алгоритмах анализа, реализованных различными инструментами. Исследования показывают, что разные системы могут выявлять различные типы нарушений доступности, что приводит к расхождениям в результатах оценки одного и того же веб-ресурса.

В связи с указанными ограничениями автоматические инструменты рассматриваются не как полноценная замена экспертной оценки, а как один из компонентов комплексного аудита доступности веб-ресурсов. Наиболее надёжные результаты достигаются при сочетании автоматических методов анализа и ручного аудита интерфейса.

Отдельной методологической проблемой автоматического тестирования является обеспечение воспроизводимости результатов анализа. Результаты проверки доступности веб-страницы могут существенно зависеть от условий выполнения анализа, включая используемый браузер, состояние страницы в момент загрузки, а также наличие динамически подгружаемых элементов интерфейса.

Современные веб-сайты часто используют асинхронную загрузку контента, динамическую генерацию интерфейсных компонентов и различные клиентские сценарии взаимодействия. В таких условиях структура DOM-документа может изменяться в процессе загрузки страницы. Если анализ выполняется до завершения всех процессов рендеринга, часть элементов интерфейса может отсутствовать в момент проверки, что приводит к искажению результатов анализа доступности.

Дополнительным фактором, влияющим на воспроизводимость анализа, является различие в механизмах работы автоматических инструментов. Некоторые системы выполняют проверку непосредственно в браузерной среде, анализируя текущую структуру DOM, тогда как другие инструменты используют предварительно сохранённую версию страницы или выполняют анализ на уровне исходного HTML-кода. Различия в подходах к обработке страницы могут приводить к различиям в числе обнаруженных нарушений.

В связи с этим при проведении сравнительных исследований доступности веб-ресурсов важным является использование единых условий анализа для всех исследуемых страниц. Это включает использование одного и того же браузерного окружения, единых правил ожидания завершения загрузки страницы и одинаковых инструментов автоматической проверки.

В рамках настоящего исследования для обеспечения сопоставимости результатов анализ страниц выполнялся в единой браузерной среде с использованием одинаковых инструментов автоматического тестирования и единых параметров анализа DOM-структуры. Такой подход позволяет минимизировать влияние внешних факторов на результаты проверки доступности и обеспечивает корректность последующего статистического анализа полученных данных. При проведении теста следует учитывать факторы, которые могут исказить собираемые данные и изменять список корректно обработанных страниц, таблица 2.

Таблица 2 — Факторы, влияющие на результаты автоматического анализа доступности

Фактор	Влияние на анализ

динамическая загрузка контента	изменение структуры DOM в момент анализа
блокировка автоматических запросов	невозможность загрузки страницы
различие версий инструментов анализа	различия в обнаруживаемых нарушениях
состояние страницы в момент проверки	неполная загрузка интерфейса
браузер и его версии	различия в отображении страниц и работе скриптов

Указанные факторы необходимо учитывать при проведении массовых исследований доступности веб-ресурсов, поскольку они могут оказывать существенное влияние на воспроизводимость результатов автоматического анализа.

2.2 Ручной аудит и экспертная интерпретация нарушений

Таким образом, автоматические инструменты тестирования позволяют эффективно выявлять формализуемые нарушения стандартов доступности и проводить масштабный анализ веб-ресурсов. Однако ограничения автоматических методов, связанные с невозможностью интерпретации контекста использования интерфейса и зависимостью результатов анализа от технических условий загрузки страниц, делают необходимым применение дополнительных методов оценки доступности. В связи с этим в практике аудита веб-ресурсов автоматический анализ обычно дополняется экспертной оценкой интерфейса, выполняемой в рамках ручного аудита [28].

Ручной аудит доступности представляет собой экспертную оценку веб-ресурсов с точки зрения соответствия стандартам доступности и удобства использования для пользователей с ограниченными возможностями здоровья. В отличие от автоматического тестирования, ручной аудит позволяет выявлять проблемы, связанные с контекстом использования интерфейса и качеством взаимодействия пользователя с веб-страницей.

Ручной аудит обычно включает анализ структуры веб-страницы, логики навигации, корректности использования текстовых альтернатив, доступности форм ввода данных, а также взаимодействия веб-ресурса со вспомогательными технологиями. В ходе экспертной оценки устанавливается степень соответствия ресурса международным стандартам WCAG, а также специфическим требованиям к государственным информационным системам РФ, изложенным в Приложении А.

Одним из ключевых методов ручного аудита является использование вспомогательных технологий, таких как программы экранного доступа, экранные дикторы. Тестирование веб-страницы с использованием экранного диктора позволяет определить, насколько корректно веб-

контент воспринимается пользователями с нарушениями зрения. Также применяется тестирование навигации с использованием клавиатуры, позволяющее оценить возможность управления веб-интерфейсом без использования мыши.

В качестве инструмента для проведения ручного аудита используются программы экранного доступа (программы экранного доступа). Эти программы предназначены для озвучивания содержимого интерфейса и позволяют пользователю взаимодействовать с операционной системой и веб-страницами без использования зрительного восприятия. Экранные дикторы в этих приложениях интерпретируют структуру документа, включая заголовки, ссылки, элементы форм и альтернативные текстовые описания изображений, и преобразуют эту информацию в синтезированную речь или вывод на брайлевский дисплей. Таким образом, использование программ экранного доступа позволяет оценить, насколько корректно реализована семантическая структура страницы и насколько удобна навигация по интерфейсу для пользователей с нарушениями зрения. Наиболее распространёнными программами экранного доступа, используемыми на персональных компьютерах, являются «NVDA», «JAWS» и «VoiceOver», их сравнение приведено в таблице 3.

Таблица 3 — Сравнение программ экранного доступа

Программа	Платформа	Преимущества	Ограничения
«NVDA» (невизуальный доступ к рабочему столу)	«Windows»	Бесплатная, открытый код, регулярные обновления, активное сообщество пользователей	Ограниченная поддержка некоторых корпоративных решений
«JAWS» (доступ к работе с помощью речи)	«Windows»	Широкая функциональность, высокая совместимость с корпоративным ПО	Коммерческая лицензия, высокая стоимость
«VoiceOver»	«macOS»	Встроена в операционную систему, стабильная интеграция с системой	Ограничена экосистемой «Apple»

По наблюдениям автора, полученным в ходе участия в научных конференциях, учебного процесса в университете и практической работы с пользователями с нарушениями зрения, наиболее часто используемой программой экранного доступа на персональных компьютерах является «NVDA». Данная программа широко применяется благодаря своей доступности, поддержке различных веб-браузеров и активному развитию со стороны сообщества разработчиков. Кроме того,

«NVDA» активно используется в образовательной среде, что делает её одним из наиболее распространённых инструментов взаимодействия с веб-контентом среди пользователей с нарушениями зрения.

В рамках настоящего исследования для проведения ручного аудита была выбрана программа экранного доступа «NVDA». Выбор обусловлен рядом факторов: программа распространяется бесплатно, регулярно обновляется и обладает широкими возможностями для анализа структуры веб-страниц. Кроме того, «NVDA» поддерживает основные веб-браузеры и может использоваться на стандартных пользовательских устройствах без необходимости приобретения специализированного программного обеспечения. Это позволяет воспроизводить условия реального взаимодействия пользователей с нарушениями зрения с веб-интерфейсами и получать более достоверные результаты оценки доступности.

В ряде исследований отмечается, что ручной аудит позволяет выявлять проблемы доступности, которые невозможно обнаружить автоматическими инструментами. К таким проблемам относятся некорректные текстовые описания изображений, сложность навигации, перегруженность интерфейса и другие факторы, влияющие на удобство использования веб-ресурсов [9].

Ручной аудит часто используется как завершающий этап комплексной оценки доступности веб-ресурсов. Он позволяет уточнить результаты автоматического тестирования и выявить наиболее критические проблемы интерфейса. В некоторых случаях результаты ручного аудита используются для классификации выявленных нарушений по уровню их влияния на пользователей.

Таким образом, ручной аудит является важным компонентом комплексной оценки доступности веб-ресурсов. Сочетание автоматических инструментов анализа и экспертной оценки позволяет получить наиболее полное представление о состоянии доступности веб-сайта и выявить ключевые проблемы интерфейса. Практические наблюдения стали основой разработки комплексного метода тестирования сайтов, далее описываемый как компонентно-инструментальный или компонентный.

2.3 Компонентный подход к оценке пользовательской значимости ошибок

Автоматизированные инструменты аудита доступности позволяют выявлять нарушения, связанные с HTML-разметкой, ARIA-атрибутами, контрастностью, доступными именами элементов, структурой заголовков и другими формальными признаками веб-страницы. Результат такой проверки представлен в виде набора правил, DOM-узлов и уровней критичности. Для практической оценки доступности этого недостаточно, поскольку пользователь взаимодействует не с отдельными DOM-узлами, а с функциональными компонентами интерфейса.

Под интерфейсным компонентом в данной работе понимается функционально завершённый элемент или группа элементов веб-страницы, предназначенная для выполнения определённой пользовательской задачи. К таким компонентам относятся кнопки, ссылки, формы, поля ввода, навигационные меню, карточки, списки, модальные окна, выпадающие элементы, шапка сайта, подвал, блок поиска и другие части интерфейса. Компонент может состоять из одного HTML-элемента или включать несколько вложенных элементов, объединённых общей пользовательской функцией.

Компонентный подход предполагает, что ошибка доступности должна рассматриваться не только как формальное нарушение правила, но и как нарушение, расположенное в определённом пользовательском контексте. Для этого необходимо определить, к какому компоненту относится ошибка, какую функцию выполняет этот компонент, является ли он интерактивным, входит ли он в основной пользовательский сценарий и может ли нарушение препятствовать восприятию информации или выполнению действия.

Такой подход особенно важен при интерпретации результатов axe-core. Данный инструмент присваивает нарушениям уровни критичности: «minor», «moderate», «serious» и «critical». Однако эти уровни отражают формальную тяжесть нарушения, а не всегда фактическую значимость ошибки для пользовательского сценария. Поэтому автоматическая критичность рассматривается как предварительная оценка, требующая экспертной интерпретации.

2.4 Практические проблемы массового аудита реальных веб-сайтов

При проведении массового аудита веб-сайтов и автоматизированного сбора данных возникают технические ограничения, связанные с программным доступом к страницам, нестабильностью их состояния и особенностями работы инструментов аудита. В рамках данного исследования были рассмотрены различные подходы к открытию и анализу страниц с использованием языка программирования Python, включая библиотеки Selenium, Playwright и работу через локальный браузер. В финальной версии системы использовалась локальная копия браузера Chrome, управляемая через библиотеку Playwright, поскольку такой подход позволил совмещать автоматизированный сбор данных с визуальным контролем состояния страницы оператором.

Одной из распространённых проблем массового аудита является обнаружение автоматических запросов веб-сайтами. Некоторые ресурсы интерпретируют повторяющиеся обращения как активность ботов или попытку массового сканирования и ограничивают доступ к содержимому. В исследуемой выборке для 3 из 141 сайтов при обращении загружалась промежуточная страница ожидания. Без визуального контроля в подобных случаях невозможно достоверно определить, какая именно версия страницы была использована для последующего

анализа: исходная страница сайта, страница ожидания, страница ошибки или модифицированный ответ системы защиты.

Дополнительные сложности возникают при работе с государственными сайтами Российской Федерации. Часть ресурсов использует устаревшие или некорректно настроенные протоколы шифрования, вследствие чего современные браузеры помечают такие страницы как потенциально небезопасные. В автоматическом режиме такие страницы могут не загружаться или прерывать сценарий анализа. Для обхода данной проблемы в исследовательском конвейере использовалось игнорирование ошибок сертификатов безопасности. Следует отметить, что применение подобной настройки допустимо только при работе с заранее проверенным и доверенным списком адресов, поскольку в обычных пользовательских сценариях игнорирование ошибок сертификатов может снижать уровень безопасности.

Существенное влияние на процесс сбора данных оказывает динамическая загрузка контента. Многие современные сайты формируют страницу не только в момент первоначальной загрузки HTML-документа, но и после выполнения JavaScript-кода. Отдельные элементы интерфейса могут появляться асинхронно, изменяться после первоначального рендеринга DOM или загружаться только после прокрутки страницы. В таких условиях автоматический алгоритм ожидания стабилизации страницы может завершаться ошибкой или, наоборот, неопределённо долго ожидать завершения сетевой активности. Поэтому для корректного сбора данных оператор должен иметь возможность контролировать момент, когда страница достигла состояния, пригодного для анализа.

Для решения этой проблемы в финальной конфигурации системы использовалась комбинация «Playwright Inspector» и функции «await page.pause()». Такой механизм позволял временно останавливать выполнение сценария без закрытия страницы и без прекращения динамических процессов внутри браузера. Во время паузы продолжалась загрузка асинхронного контента, а оператор мог визуально проверить состояние страницы, дождаться стабилизации интерфейса, зафиксировать ошибку загрузки или принять решение о продолжении анализа.

Взаимодействие с оператором осуществлялось через терминал среды разработки. После остановки сценария оператор передавал управляющую команду: продолжить выполнение сценария и перейти к сбору данных либо зафиксировать ошибку загрузки и перейти к следующему сайту. Такой полуавтоматический режим предотвращает сбор данных с некорректно отображённых страниц и позволял фиксировать случаи, когда автоматическая обработка не отражает фактическое состояние интерфейса.

Отдельную проблему представляют элементы, не относящиеся к основному содержимому страницы: модальные окна согласия на использование куки-файлов, формы подписки, всплывающие уведомления, баннеры и другие всплывающие компоненты. Такие элементы могут изменять DOM-структуру страницы, перекрывать основной интерфейс и влиять на результаты

аудита доступности, хотя не всегда являются частью основного пользовательского сценария. Массовое отключение подобных элементов с помощью универсального скрипта затруднено, поскольку они имеют различную DOM-структуру, появляются в разные моменты времени и реализуются разными механизмами.

Теоретически возможно автоматизировать закрытие подобных элементов путём записи действий оператора и последующего воспроизведения кликов средствами браузерной автоматизации. Однако такой подход требует индивидуальной настройки для каждой страницы и снижает воспроизводимость исследования. Кроме того, некоторые операции взаимодействия со страницей ограничиваются особенностями «CDP (Chrome DevTools Protocol)», что затрудняет универсальное управление всплывающими компонентами в условиях массового аудита.

После стабилизации страницы выполнялся сбор DOM-структуры и анализ доступности с использованием axe-core. Данный анализ осуществлялся путём внедрения JavaScript-кода в страницу. Однако на ряде сайтов выполнение пользовательского JavaScript может блокироваться политиками безопасности, особенностями окружения или настройками страницы. В таких случаях автоматический аудит может завершиться неполно, не выполниться вовсе или дать результат, не отражающий фактическую доступность анализируемого интерфейса.

Одним из способов частичного обхода подобных ограничений является сохранение состояния страницы в формате MHTML. Такой подход позволяет зафиксировать представление страницы на момент анализа и использовать сохранённую копию для последующего извлечения данных. Однако MHTML-копии также имеют ограничения: внешние скрипты, динамические ресурсы и некоторые состояния интерфейса могут сохраняться частично. Поэтому использование MHTML в исследовании рассматривается не как идеальная замена работе с живой страницей, а как технический механизм фиксации состояния страницы и извлечения метрик в условиях ограниченной автоматизации.

Дополнительно для получения метрик доступности использовалось браузерное расширение WAVE инструмент оценки веб-доступности. Использование расширений в автоматизированной среде связано с отдельными ограничениями. Подключение расширений возможно только при работе с локальным пользовательским профилем браузера, тогда как в стандартных режимах автоматизации загрузка пользовательских расширений часто ограничена или нестабильна. Кроме того, запуск расширения WAVE невозможно полностью автоматизировать, поскольку его интерфейс активируется через всплывающее окно панели инструментов браузера, требующее пользовательского действия.

После активации WAVE результаты анализа внедряются в структуру страницы, в том числе через дополнительные элементы интерфейса расширения. Это затрудняет прямое считывание результатов стандартными методами анализа DOM. Для обхода данного ограничения в

исследовании использовалось сохранение страницы в формате MHTML после запуска WAVE, после чего из сохранённого документа извлекались числовые показатели отчёта. Преимуществом такого подхода является возможность получить сопоставимые метрики WAVE/AIM для большого набора страниц, однако его применение требует контроля корректности состояния страницы и наличия результатов анализа в сохранённом документе.

В результате описанных технических ограничений процесс массового аудита доступности реальных веб-сайтов не может рассматриваться как полностью автоматический. В рамках исследования он был организован как контролируемый полуавтоматический процесс, сочетающий программный сбор данных, визуальную проверку состояния страницы оператором, фиксацию ошибок загрузки, получение DOM-, axe-core- и WAVE/AIM-метрик, а также последующую обработку результатов. Анализ 278 страниц занимал приблизительно 4–6 часов. При сравнении обычных версий и версий для слабовидящих страниц важно минимизировать временной интервал между сбором данных, поскольку содержимое сайтов может обновляться, например за счёт новостных блоков, что делает данные, собранные в разные моменты времени, потенциально несопоставимыми.

Таким образом, практические проблемы массового аудита показывают, что достоверность результатов автоматизированного анализа зависит не только от выбранного инструмента, но и от качества контроля состояния страницы. Поэтому в настоящем исследовании технические статусы, ошибки загрузки, особенности выполнения JavaScript и возможность получения стабильной копии страницы рассматриваются как значимые элементы методики аудита, влияющие на интерпретацию полученных данных.

Отдельное ограничение массового аудита связано с потерей сведений об интерфейсных компонентах из-за некорректной или недостаточно семантической разметки. Автоматизированные инструменты и скрипты сбора данных обычно формируют перечень элементов на основе HTML-тегов, ролей, атрибутов и DOM-структуры. Если компонент визуально выполняет определённую функцию, но реализован с использованием неподходящих элементов или без корректных «ARIA»-атрибутов, он может не попасть в соответствующую категорию при автоматической классификации. Например, визуальная кнопка, реализованная как «div» без роли «button», может не учитываться как кнопка; навигационный блок без «nav» или соответствующего «role» может не попасть в группу навигационных элементов; поле ввода без корректной связи с «label» может быть формально обнаружено как элемент формы, но его пользовательская функция и семантическая ошибка будут интерпретированы неполно. В результате при массовом аудите часть компонентной структуры страницы оказывается скрытой для автоматического анализа: данные о таких компонентах не исчезают из DOM, но теряются как сведения о функциональных элементах

интерфейса. Это ограничивает точность количественных метрик и дополнительно обосновывает необходимость ручной компонентной верификации.

2.5 Метрики доступности и структурной сложности веб-страниц

Оценка доступности веб-ресурсов требует использования количественных показателей, позволяющих формализовать результаты аудита и проводить сравнительный анализ различных веб-страниц. Метрики доступности позволяют представить результаты анализа в виде числовых характеристик, отражающих состояние интерфейса с точки зрения соответствия рекомендациям стандартов WCAG. Использование подобных показателей делает возможным применение статистических методов анализа, а также проведение масштабных исследований доступности веб-ресурсов.

В научных исследованиях предлагаются различные подходы к количественной оценке доступности веб-контента. Наиболее распространённым является анализ количества выявленных нарушений доступности и их соотношения с общим количеством элементов веб-страницы. Такой подход позволяет учитывать различия в сложности интерфейса и объёме содержимого страниц, что особенно важно при сравнении сайтов различной структуры и размера.

Метрики доступности могут формироваться на основе данных автоматических инструментов тестирования, таких как WAVE и axe-core. Эти инструменты выполняют анализ структуры HTML-документа и выявляют нарушения стандартов WCAG, связанные с использованием элементов интерфейса, семантической разметки и параметров визуального отображения контента. В результате анализа формируются отчёты, содержащие информацию о типах обнаруженных проблем, их количестве и предполагаемой степени влияния на пользователей.

Метрики, получаемые в ходе автоматического анализа доступности, можно условно разделить на несколько групп. К первой группе относятся показатели, характеризующие количество обнаруженных ошибок доступности. Под данным показателем понимается число нарушений требований стандартов WCAG, выявленных в процессе автоматического анализа страницы. Такие нарушения могут включать отсутствие альтернативных текстов у изображений, проблемы цветового контраста, некорректное использование ARIA-атрибутов, ошибки структуры заголовков и другие нарушения требований доступности.

Помимо абсолютного числа ошибок в исследованиях доступности часто используются агрегированные показатели. Некоторые инструменты автоматического анализа формируют интегральные индексы доступности страницы, позволяющие оценить общее качество интерфейса. В частности, инструмент WAVE предоставляет показатель AIM-Score, отражающий обобщённую оценку доступности веб-страницы. Данный показатель рассчитывается на основе совокупности

обнаруженных проблем и используется как интегральная характеристика уровня доступности страницы.

Одной из проблем прямого сравнения количества ошибок доступности является различие в размере и сложности веб-страниц. Более крупные страницы обычно содержат больше элементов интерфейса, что может приводить к увеличению числа выявленных ошибок. Для устранения данного эффекта используются нормированные метрики доступности.

Нормированные показатели рассчитываются как отношение числа ошибок доступности к общему количеству элементов веб-страницы. В качестве базового значения может использоваться количество элементов DOM (объектная модель документа), представляющее структуру веб-документа. В этом случае показатель доступности может быть выражен следующим образом:

$$Accessibility\ Ratio\ =\ N_{errors}\ /\ N_{DOM}\ , \qquad (1)$$

N_{errors} — количество выявленных ошибок доступности,

N_{DOM} — общее число элементов DOM на странице.

Использование нормированных показателей позволяет корректно сравнивать страницы различной сложности и размера, поскольку учитывает различия в количестве элементов интерфейса.

Помимо анализа ошибок доступности в исследованиях также используется анализ структуры веб-страницы. В рамках данного подхода рассматриваются характеристики интерфейса, связанные с количеством различных типов элементов страницы. К таким элементам относятся изображения, ссылки, кнопки, формы ввода данных, заголовки различных уровней и структурные элементы навигации. Подобные показатели позволяют оценить сложность интерфейса и определить, какие элементы структуры страницы могут влиять на уровень её доступности.

Особое внимание при анализе доступности веб-страниц уделяется ошибкам, связанным с использованием изображений. Автоматические инструменты тестирования фиксируют наличие альтернативного текста у изображения как формальное требование стандартов WCAG. Однако реальное влияние отсутствия альтернативного описания на доступность страницы может существенно зависеть от контекста использования изображения.

В связи с этим при анализе доступности веб-ресурсов целесообразно проводить оценку характеристик не только по всем компонентам страниц, но и контекстно отсекать ошибки, связанные с изображениями, так как ограничение визуального контента используется как самая распространённая методика увеличения показателя доступности страницы, AIM-Score. Такой подход позволяет более корректно интерпретировать результаты анализа и снижает вероятность искажения статистических выводов за счёт исключения большого числа однотипных ошибок.

В рамках настоящего исследования данная проблема учитывается при формировании аналитических метрик доступности. Помимо общего числа обнаруженных нарушений используется показатель количества ошибок без учёта нарушений, связанных с изображениями. Перечень метрик, агрегируемых при обработке данных и записываемых в таблицу «raw_pages.csv», приведён в приложении Б. Это позволяет отдельно анализировать влияние изображений на общий уровень доступности страницы и более корректно интерпретировать результаты статистического анализа.

Таким образом, использование совокупности различных показателей — количества ошибок доступности, их классификации по степени серьёзности, интегральных индексов доступности и структурных характеристик веб-страницы — позволяет получить более полное представление о состоянии доступности веб-ресурсов. Комплексный подход к формированию метрик обеспечивает возможность проведения сравнительного анализа различных веб-страниц и выявления закономерностей, связанных с уровнем их доступности.

ГЛАВА 3. РАЗРАБОТКА И АПРОБАЦИЯ КОМПОНЕНТНО-ИНСТРУМЕНТАЛЬНОГО ПОДХОДА К АУДИТУ ДОСТУПНОСТИ

3.1 Исходные данные и структура набора веб-страниц

На первом этапе исследования для анализа был собран список государственных сайтов. Для каждого сайта было проверено наличие версии для слабовидящих и его тип. Это было необходимо для дальнейшего этапа попарного сравнения обычной версии страницы и версии для слабовидящих. В процессе работы каждый сайт был проверен на работоспособность, после чего фиксировался его URL и тип интерфейса для слабовидящих.

В результате сбора адресов государственных сайтов было сохранено 232 адреса из которых 20 стабильно не открывались, а один при открытии имел явную ошибку при загрузке, вместо интерфейса показывался текстом фрагмент HTML. Таким образом в тестировании изначально могли принять участие 211 адресов. Из них:

- 3 сайта не реагировали на переход к версии для слабовидящих.
- 34 сайта инициировали переход на страницу для слабовидящих с новым URL.
- у 127 сайтов версии для слабовидящих активировалась на исходной странице, изменения происходили с помощью JS скрипта.

В некоторых вариантах адаптации сайтов для пользователей с нарушениями зрения наблюдались изменения визуального представления контента, а также появление панелей настройки поверх основного интерфейса сайта. В других случаях фиксировались лишь отдельные изменения визуальных элементов интерфейса. В ходе анализа страниц было установлено, что схожие подходы к адаптации чаще встречались внутри одного субъекта Российской Федерации, однако формализованный анализ и классификация используемых решений в рамках данного исследования не проводились.

Сохранение URL происходило в специально подготовленный файл, где хранились только актуальные адреса, которые прошли проверку на корректную загрузку. В случае ошибок при загрузке страницы (например, 404 или проблемы с редиректами) сайт исключался из анализа. Данные по ошибкам, возникавшим при аудите, сохранялись в формате JSON и использовались для дальнейшего анализа. Так как состояние загрузки было нестабильным, этап проверки сайта на работоспособность был интегрирован в разработанные программы для аудита страниц, чтобы при каждом аудите не исключать сайты, которые были временно недоступны во время предыдущего сканирования состояний.

После завершения массового аудита все данные о состоянии страниц были свёрнуты и сохранены в таблицах в формате «CSV», включая все метрики структуры и доступности для каждой версии страницы. При повреждении данного файла можно извлечь данные путём парсинга

метаданных «JSON» файлов, даже в случае провала загрузки данные дополнительно сохранялись в соответствующий файл. Табличное представление создаётся для удобства сравнения аудитов.

В результате агрегации было сформировано 278 записей об отдельных страницах: обычных версиях и версиях для слабовидящих. На их основе было выделено 143 потенциальные пары страниц, объединённые по исходному URL. Из них 8 пар были неполными, поскольку отсутствовала одна из версий страницы, а 14 пар были исключены из-за ошибок загрузки хотя бы одной версии. В итоговый статистический анализ вошла 121 корректная пара страниц.

Общая схема сбора и обработки данных в рамках исследования предполагает использование контролируемого конвейера для извлечения метрик доступности и структурных характеристик веб-страниц. Включенные в анализ веб-ресурсы имеют две версии интерфейса: стандартную и адаптированную для пользователей со сниженным зрением. Для каждой страницы фиксируется её исходный адрес, тип интерфейса и статус загрузки.

Для сбора данных с веб-страниц использовался специально разработанный программный конвейер, который обеспечивал автоматизированную загрузку страниц, сохранение данных при ошибках навигации, загрузки, ожидание стабилизации страниц с дальнейшим сохранением в формате MHTML, а также извлечение структурных метрик и результатов анализа доступности с помощью инструментов axe-core и WAVE. Подробнее о процессе сбора данных и применяемых инструментах см. в разделе 3.2.

Конвейер начинается с автоматизированной загрузки веб-страницы в браузерной среде, управляемой через локальный экземпляр браузера, с использованием инструментов «Playwright». Использование «Playwright» обеспечило возможность имитации реального пользовательского поведения, что позволило более точно зафиксировать состояние страницы в момент сбора данных. Такой подход позволял анализировать фактическое состояние DOM, сформированное после обработки HTML, CSS и JavaScript, а также учитывать элементы, появившиеся в результате динамической загрузки.

В процессе загрузки страниц особое внимание уделялось выявлению и учёту ошибок, связанных с невозможностью загрузки страницы или её части. Среди таких ошибок были:

- Ошибки сети — например, невозможность установить соединение с сервером, ошибки 404 или 500;
- Ошибки перенаправлений — неправильные редиректы, зацикленные перенаправления;
- Блокировки автоматических запросов со стороны веб-сайтов, интерпретирующих их как подозрительную активность — например, блокировки по «IP» или с использованием «CAPTCHA»;
- Ошибки из-за динамического контента — например, незагруженные изображения, скрипты или элементы интерфейса из-за асинхронной загрузки или проблем с сервером;

- Ошибки JavaScript — связанные с невозможностью корректной работы скриптов или их блокировки, которые могут привести к частичной или полной неработоспособности страницы.

Эти ошибки фиксировались в специальном поле «navigation_error», что позволяло сохранять данные о страницах с проблемами загрузки в отдельную категорию. В случае выявления таких ошибок процесс анализа страницы прекращался, а информация о проблеме сохранялась для дальнейшего исследования.

После загрузки и стабилизации страницы в браузерной среде происходило сохранение страницы в MHTML, сбор DOM-структуры и анализ axe-core. Иными словами, структурные характеристики страницы и нарушения доступности, выявленные axe-core, извлекались не из сохранённой параллельно MHTML-копии, а из страницы, открытой в браузере после выполнения необходимых сценариев загрузки. Такой подход позволяет избегать искажения данных о страницах при автоматическом сборе ошибок, но сохраняет разметку и стили для возможной переоценки.

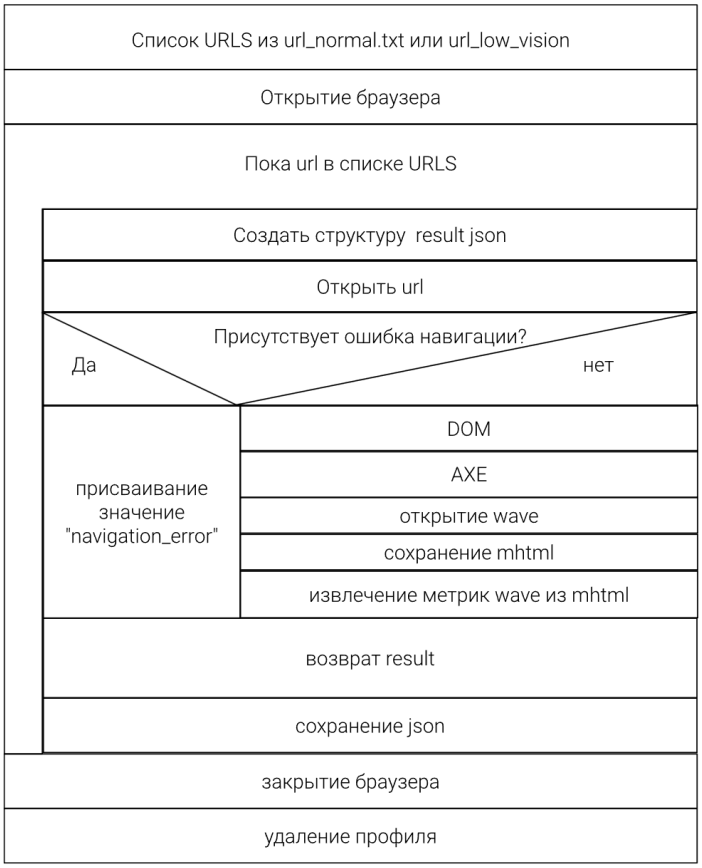
Кроме того, в конвейере используется интеграция с расширением WAVE для оценки доступности. Для каждого из сайтов, кроме анализа DOM-метрик и стандартных ошибок, также извлекаются метрики из WAVE, включая AIM-Score и другие показатели доступности. Эта информация комбинируется с данными, полученными с помощью axe-core, и сохраняется в итоговых таблицах для дальнейшего статистического анализа.

Основной задачей этапа сбора данных является обеспечение воспроизводимости. Все извлечённые данные сохраняются в структурированном виде в формате JSON, что позволяет точно фиксировать каждый элемент интерфейса и выявленные нарушения доступности. Использование MHTML позволяет гарантировать, что в случае фрагментации данных будет альтернативный путь повторного обращения к исследуемой странице в том состоянии, в каком она была на момент аудита. А в случае снятия MHTML копии после запуска расширения WAVE мы получаем доступ к содержимому «sidebar-container» в котором хранятся данные WAVE. Прямые запросы из Playwright к «sidebar» и к интерфейсу расширения ограничены архитектурой Chrome и моделью изоляции расширений: эти области не являются частью основного DOM страницы, а потому недоступны для обычного обхода через стандартные методы взаимодействия с документом.

После извлечения всех показателей данные объединяются в агрегированную запись, соответствующую отдельной веб-странице. Каждая запись включает идентификатор сайта, URL страницы, тип версии интерфейса и рассчитанные метрики структуры и доступности. Пример формирования такой записи приведён в приложении Б. Полученные записи агрегируются в сводную таблицу «raw_pages.csv», где каждая строка соответствует отдельной странице. Эта таблица используется как основной источник данных для последующего статистического анализа.

Таким образом, этап сбора данных включает в себя автоматизированный процесс загрузки и стабилизации страниц с помощью «Playwright», извлечение данных о структуре DOM и проверку доступности с помощью axe-core и WAVE. Все собранные данные сохраняются в структурированном формате для дальнейшего статистического анализа, что позволяет провести сравнение между версиями обычная версия и версия для слабовидящих, а также проверить статистические гипотезы.

Рисунок 1 — План обработки списка сайтов, схема Насси-Шнейдермана



3.2 Программный конвейер сбора целевых показателей

В данном разделе подробно рассматривается программный конвейер, использованный для сбора и анализа данных о доступности веб-страниц. Этот конвейер состоит из последовательных этапов, каждый из которых выполняет конкретную задачу по сбору данных о структуре страниц и нарушениях доступности с использованием инструментов «Playwright», axe-core и WAVE. Основная цель этого конвейера — автоматизация сбора метрик, которые затем используются для статистического анализа доступности веб-страниц как для обычных версий интерфейсов, так и для адаптированных версий для слабовидящих.

Каждый этап в рамках этого конвейера выполняет свою функцию и позволяет получать результаты, которые могут быть использованы для более глубокого анализа доступности. Конвейер

также обеспечивает возможность масштабируемости и расширения, позволяя легко добавлять новые сценарии аудита.

Программный конвейер сбора данных был реализован в скрипте «a11y_info_aggregation.py». Он выполнял загрузку страницы в браузерной среде, сбор DOM-структуры с учётом видимости элементов, запуск axe-core, сохранение результатов WAVE/AIM через MHTML и формирование JSON-записи для каждой страницы. В структуре результата фиксировались блоки «meta, «navigation_error», «dom», «elements», «axe», «manual», «cdp и «wave».

3.2.1 Проверка функционирования с помощью библиотеки Playwright

Для проверки функционирования страниц используется библиотека языка Python, Playwright, которая обеспечивает автоматическую загрузку и стабильную работу с веб-страницами. Важно, чтобы веб-сайт был не только доступен для пользователей, но и функционально работоспособен. Веб-ресурсы, не обеспечивающие доступность в случае ошибок загрузки или неправильного отображения, могут привести к недоступности важной информации, например, контактных данных организации.

В текущем контексте функциональная доступность является частью анализа веб-метрик, данный параметр встречается в интегрируемых на страницы сервисах для мониторинга активности. При работе государственных сайтов важно иметь механизм отслеживания функциональной доступности списка страниц для своевременного обнаружения сбоев сервисов и отправки их на технический аудит.

Для исключения недоступных страниц в конвейере используется контроль времени загрузки страницы с помощью «Playwright». В случае, если страница не загружается в установленный срок, например, «TimeoutError»:

await page.goto(url, wait_until="load", timeout=5000) # Тайм-аут 5 секунд

Если процесс загрузки не завершился за отведённое время, то страница сохраняется с указанием ошибки.

Ошибка «PlaywrightError» является общей ошибкой, которая может возникнуть при любых проблемах, связанных с инструментом «Playwright». Это может быть связано с некорректным использованием «API Playwright», ошибками взаимодействия с браузером или другими внутренними ошибками.

3.2.2 Ожидание стабилизации страницы и обработка динамического контента

Следующим важным этапом является ожидание стабилизации контента на странице, особенно в случае динамически загружаемого контента через JavaScript. Для этого используется асинхронная функция «async def get_stable_page_content».

Современные веб-страницы часто используют динамическую загрузку контента, который появляется только после полной загрузки страницы. Чтобы удостовериться в полном содержимом страницы, необходимо подождать, пока все элементы не будут загружены и отображены, так как страницы могут выглядеть визуально стабилизированными для пользователя, но программно процесс загрузки может продолжаться. В таких случаях для получения точных данных необходимо использовать дополнительные инструменты, такие как консоль разработчика в браузере, чтобы избежать ложных срабатываний.

Таким образом, «Playwright» позволяет контролировать это состояние, гарантируя, что данные, которые мы получаем, будут полными и точными.

3.2.3 Сохранение страницы в формате MHTML с использованием Playwright

После того как страница стабилизирована и готова к дальнейшему анализу, выполняется её сохранение в формате MHTML, код процедуры представлен листингом 1. Этот формат позволяет зафиксировать полное состояние страницы на момент её анализа, включая все элементы, стили и динамически загружаемый контент. Использование MHTML, листинг 1, обеспечивает высокую степень воспроизводимости исследования, поскольку позволяет в случае изменений на сайте или проблем с его доступностью вернуться к сохранённой версии страницы. Это предотвращает возможные искажения данных, которые могут возникнуть при повторном анализе, а также гарантирует, что анализ будет проводиться на одинаковом наборе данных.

Важно отметить, что автоматизированное сохранение страницы в формате MHTML в рамках исследования выполнялось через Chrome DevTools Protocol (CDP), поддерживаемый браузерами на базе Chromium. Поэтому корректность работы данного механизма зависит от используемого браузерного движка и средств автоматизации. При проведении повторной оценки страницы на основании копий необходимо убедиться в их сохранности копии так как при проведении эксперимента наблюдались существенные расхождения с оригинальными страницами. Предположительно это происходит из-за ограничений доступа к содержимому со стороны сохраняемого ресурса.

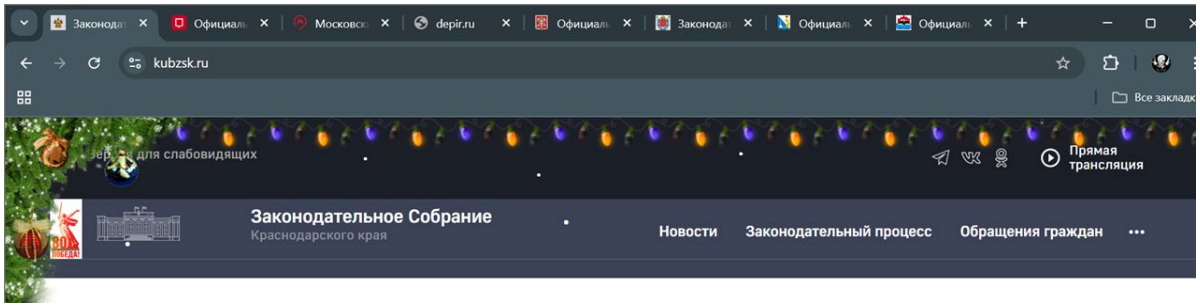
Листинг 1 — Сохранение веб-страницы в формате MHTML

```
from pathlib import Path
from datetime import datetime, timezone
import hashlib
# Папка запуска
RUN_DIR = Path("results_" + datetime.now(timezone.utc).strftime("%Y%m%d_%H%M%S"))
```

```
MHTML_DIR = RUN_DIR / "mhtml"
MHTML_DIR.mkdir(parents=True, exist_ok=True)
MODE = "low_vision"
def md5_hash(s: str) -> str:
    return hashlib.md5(s.encode("utf-8")).hexdigest()[:8]
# ---------------- Сохранение страницы в MHTML ----------------
try:
    client = await page.context.new_cdp_session(page)
    snapshot = await client.send(
        "Page.captureSnapshot",
        {"format": "mhtml"}
    )
    mhtml_raw_data = snapshot.get("data", "") or ""
    mhtml_path = MHTML_DIR / f"{md5_hash(url)}_wave_{MODE}.mhtml"
    mhtml_path.write_text(
        mhtml_raw_data,
        encoding="utf-8",
        errors="ignore",
        newline="\n"
    )
except Exception as e:
    print(f"Ошибка при сохранении MHTML: {e}")
```

При повторной проверке реальных страниц было выявлено, что динамические блоки, прежде всего новостные ленты, элементы-декораторы, рисунок 2, могут изменять состав DOM между запусками аудита. Поэтому для сопоставимости данных в системе имён файлов фиксировался момент сбора, а в общем файле аудита страницы дополнительно отражалось состояние загрузки страницы, наличие технических ошибок. Эти сведения позволяют позднее отличать устойчивые проблемы доступности от временных изменений содержимого.

Рисунок 2 — Пример временного компонента оформления сайта



Также исходные данные о состоянии страниц нужны для сравнения системы после исправления ошибок с предыдущим состоянием, переоценки выводов в случае разногласий в трактовке ошибок.

3.2.4 Извлечение HTML из MHTML

Для анализа веб-страниц, сохранённых в формате MHTML, важным этапом является извлечение HTML-кода. Этот процесс позволяет преодолеть ограничения архитектуры современных браузеров, в частности «Chromium», где усиливается разделение между компонентами, созданными расширениями, и основным контентом страницы. В частности, наложенные ограничения на прямое извлечение данных из sidebar компонентов могут усложнить анализ, если полагаться только на стандартные средства браузера.

В отличие от простого сохранения страницы в формате HTML, MHTML («MIME HTML») представляет собой архив, включающий не только код разметки, но и связанные с ним ресурсы, такие как изображения, стили и другие элементы. Это позволяет сохранить полное состояние страницы на момент её анализа, включая все визуальные и динамические компоненты, что критически важно для исследования доступности.

После сохранения страницы в формате MHTML необходимо извлечь HTML-код, содержащий структуру страницы, для дальнейшего анализа доступности с использованием инструментов, таких как axe-core и WAVE.

Функция ниже разбирает MHTML как MIME-документ, проходит по его частям и извлекает только те блоки, у которых тип содержимого: "text/html", "application/xhtml+xml". На выходе получается список HTML-фрагментов "List[str]".

Листинг 2 — Извлечение HTML из MHTML

```
from typing import cast, List
from email import policy
from email.message import Message
from email.parser import BytesParser
def mhtml_to_html_candidates(mhtml_raw: str) -> List[str]:
    raw_bytes = mhtml_raw.encode("utf-8", errors="ignore")
    msg: Message = BytesParser(policy=policy.default).parsebytes(raw_bytes)
    html_parts: List[str] = []
    parts = msg.walk() if msg.is_multipart() else [msg]
    for part in parts:
        ctype = (part.get_content_type() or "").lower()
```

```
            if ctype not in ("text/html", "application/xhtml+xml"):
                continue
            payload_any = part.get_payload(decode=True)
            if payload_any is None:
                continue
            payload = cast(bytes, payload_any)
            charset = part.get_content_charset() or "utf-8"
            try:
                html_parts.append(payload.decode(charset, errors="ignore"))
            except LookupError:
                html_parts.append(payload.decode("utf-8", errors="ignore"))
        return html_parts
```

3.2.5. Сбор ключевых метрик: DOM, AXE, WAVE

Следует учитывать, что автоматический подсчёт элементов фиксирует все элементы, находящиеся в DOM и признанные видимыми на момент аудита. Поэтому в версиях для слабовидящих в подсчёт включались не только элементы основного содержимого страницы, но и компоненты панели настройки версии для слабовидящих: кнопки изменения размера шрифта, переключатели цветовой схемы, элементы управления контрастностью, межстрочным интервалом и другие служебные компоненты. Эти элементы не исключались из анализа, поскольку они являются частью пользовательского интерфейса версий для слабовидящих и также должны быть доступны пользователю.

Следует учитывать, что автоматическая классификация элементов по типам опирается на их HTML-семантику, роли ARIA и атрибуты разметки. Поэтому в соответствующие категории попадают только те компоненты, которые представлены в DOM как кнопки, ссылки, формы, навигационные области или другие распознаваемые элементы интерфейса. Если фактическая пользовательская роль компонента не соответствует его разметке, например визуальная кнопка реализована как «div» без роли «button», такой элемент может не учитываться в соответствующей категории автоматического анализа. Следовательно, часть функциональных компонентов может выпадать из компонентной классификации не потому, что они отсутствуют на странице, а из-за некорректной или недостаточно семантической реализации.

Дополнительно фиксируются признаки потенциальных проблем структуры страницы, например пропуски уровней заголовков. Такой анализ позволяет выявлять ошибки разметки, которые могут негативно влиять на доступность интерфейса для пользователей вспомогательных технологий. Пример функции обнаружения пропусков уровней заголовков приведён в листинге 4.

Листинг 4 — Агрегация структурных элементов страницы

```
headings_total = sum_int(dom, ["h1", "h2", "h3", "h4"])
headings_visible = sum_int(dom, ["h1_visible", "h2_visible", "h3_visible", "h4_visible"])
landmarks_total = sum_int(dom, ["nav", "main", "header", "footer", "section", "article"])
landmarks_visible = sum_int(dom, ["nav_visible", "main_visible", "header_visible", "footer_visible", "section_visible", "article_visible"])
```

Параллельно со структурным анализом выполняется автоматический аудит доступности страницы с использованием инструментов axe-core и WAVE. Результаты axe-core представляются в формате JSON и содержат сведения о типах нарушений и уровне их критичности. Для последующего анализа выполняется агрегация количества ошибок по встроенным категориям «critical», «serious», «moderate» и «minor». Пример соответствующей функции представлен в листинге 5.

Листинг 5 — Агрегация результатов анализа «AXE»

```
impact_summary = safe_get(axe, "impact_summary", default={}) or {}
if not isinstance(impact_summary, dict):
    impact_summary = {}
axe_minor = int(impact_summary.get("minor", 0) or 0)
axe_moderate = int(impact_summary.get("moderate", 0) or 0)
axe_serious = int(impact_summary.get("serious", 0) or 0)
axe_critical = int(impact_summary.get("critical", 0) or 0)
axe_nodes_count = to_int(axe.get("nodes_count"))
if axe_nodes_count is None:
    axe_nodes_count = len(axe_nodes)
```

Для проведения исследования был сформирован набор данных, включающий результаты автоматизированного аудита доступности веб-страниц российских сайтов. Исходная выборка была сформирована на основе списка более чем 140 веб-ресурсов. Для каждого сайта анализировались две версии веб-страницы: стандартная версия интерфейса (обычная версия) и версия страницы, предназначенная для пользователей со сниженным зрением (low_vision).

В результате автоматизированного сбора данных было сформировано несколько таблиц, содержащих результаты аудита страниц и рассчитанные показатели доступности. Основные данные сохранялись в формате «CSV».

Основной таблицей является «raw_pages.csv», содержащая результаты анализа отдельных веб-страниц. Таблица содержит 278 строк, где каждая строка соответствует одной странице в определённой версии интерфейса. Основными полями таблицы являются:

- «start_url» — адрес анализируемой веб-страницы;
- «finish_url» — адрес фактически анализируемой веб-страницы;
- «version» — тип версии страницы (normal или low_vision);
- «navigation error» — текст ошибки загрузки страницы (если страница не была корректно загружена);
- «wave_status» — служебный статус выполнения анализа WAVE;
- «aim_score» — интегральный показатель доступности страницы, рассчитываемый системой WAVE/«AIM».

Помимо показателей доступности таблица содержит структурные характеристики веб-страницы. К ним относятся показатели количества видимых элементов различных типов интерфейса:

- «dom_total_visible» — общее количество элементов интерфейса, отображаемых пользователю;
- «images_visible» — число видимых изображений;
- «links_visible» — количество ссылок;
- «buttons_visible» — количество элементов управления;
- «forms_visible» — количество элементов форм;
- «h1_visible», «h2_visible», «h3_visible», «h4_visible» — количество заголовков различных уровней;
- «landmarks_visible» — число структурных элементов страницы (nav, main, header, footer, section, article);
- «headings_visible» — общее число заголовков страницы.

Кроме структурных характеристик таблица содержит результаты автоматического анализа доступности, полученные с использованием инструмента axe-core. В частности, фиксируются следующие показатели:

- «axe_nodes_count» — число DOM-узлов, содержащих нарушения доступности;
- «axe_nodes_image_related» — число нарушений, связанных с изображениями (например отсутствие альтернативного текста или проблемы контрастности);
- «axe_nodes_noimg» — количество нарушений после исключения ошибок, связанных с изображениями.

Также сохраняется распределение нарушений по степени серьёзности: «axe_minor», «axe_moderate», «axe_serious», «axe_critical»

Дополнительно в таблице присутствует поле «manual_present», которое фиксирует наличие экспертной ручной оценки страницы. Базовая классификация серьёзности ошибок, используемая системой axe-core, соответствует рекомендациям WCAG, однако она не отражает ситуации, когда отдельные ошибки делают интерфейс полностью недоступным для пользователей вспомогательных технологий.

Для ручного аудита формируется набор показателей «manual_impact_summary», включающий следующие категории: «ruin», «critical», «serious», «moderate», «minor».

Для последующей обработки данных формировалась дополнительная агрегированная таблица «merged_by_page.csv», представляющая очищенную и агрегированную версию исходных данных. В данной таблице сохраняются основные характеристики страниц и добавляется показатель «dom_noimg_visible», отражающий количество элементов интерфейса без учёта изображений.

Для проведения сравнительного анализа между версиями страниц формировались таблицы «paired_pages.csv» и «pairs.csv». В этих таблицах каждая строка соответствует одной паре страниц (обычная версия и версия для слабовидящих), принадлежащих одному сайту. Ключом таблицы является поле start_url.

Для каждой пары формируются поля вида:

- «aim_score_normal» / «aim_score_low»
- «navigation_error_normal» / «navigation_error_low»
- «dom_total_visible_normal» / «dom_total_visible_low»

Также рассчитываются дополнительные агрегированные показатели, включая нормированные значения ошибок доступности.

Всего в исходном наборе данных было сформировано 143 потенциальных пары страниц. Однако не для всех сайтов присутствовали обе версии страницы, и часть страниц не была корректно загружена.

При подготовке данных выполнялась фильтрация по следующим условиям: исключались пары, в которых хотя бы одна версия страницы не была успешно загружена; исключались сайты, для которых отсутствовала одна из версий страницы.

После фильтрации из анализа были исключены: 14 пар страниц с ошибками загрузки; 8 неполных пар страниц.

В результате в итоговый статистический анализ были включены 121 корректная пара страниц.

Подготовка данных и выполнение статистического анализа осуществлялись с использованием языка программирования Python и библиотек анализа данных. Использование

программных средств позволило автоматизировать обработку большого объёма данных и обеспечить воспроизводимость результатов исследования.

Основными используемыми библиотеками являлись:

- «pandas» — для загрузки и обработки табличных данных;
- «SciPy» — для выполнения статистических тестов;
- «statsmodels» — для корректировки p-значений при множественных сравнениях;
- «matplotlib» — для визуализации результатов анализа.

Для снижения риска повреждений исходных файлов весь аудит разделён на отдельные блоки, которые не имеют доступа к исходным данным предыдущего шага обработки, анализируемые метрики хранились в виде «CSV»-таблиц, собранных из .json и .mhtml. Таблица «raw_pages.csv» содержит результаты аудита отдельных страниц, тогда как таблицы «merged_by_page.csv» и «paired_pages.csv» представляют агрегированные данные, используемые для дальнейшего анализа.

Перед выполнением статистического анализа из выборки исключались страницы, для которых процесс загрузки завершился ошибкой. Наличие ошибки фиксируется в поле «navigation_error». Фильтрация выполнялась следующим образом: valid_raw = raw[raw.navigation_error.isna()]

После этого из набора данных отбирались только те сайты, для которых присутствуют обе версии страницы — базовая и адаптированная для слабовидящих. Для этого выполнялась группировка по адресу страницы («start_url») и подсчёт числа различных значений поля version. Пример группировки данных показан в фрагменте кода 6.

Листинг 6 - формирование таблицы, содержащей значения метрик для двух версий страницы

```
merged_valid = merged[
    (merged.version.isin(['normal', 'low_vision'])) &
    (merged.start_url.isin(valid_pairs))
]
by_pair = merged_valid.pivot(
    index='start_url',
    columns='version',
    values=[
        'dom_total_visible',
        'headings_visible',
        'landmarks_visible',
```

```
        'links_visible',
        'buttons_visible',
        'forms_visible',
        'images_visible',
        'axe_nodes_count',
        'axe_nodes_noimg',
        'aim_score'
    ]
)
```

В результате данной операции формируется таблица, в которой каждая строка соответствует одной паре страниц, а значения метрик для различных версий страницы представлены в отдельных столбцах.

Для повышения удобства дальнейшей обработки столбцам таблицы присваивались новые имена, содержащие указание версии страницы: Подобная структура данных позволяет напрямую сравнивать значения метрик между версиями страниц и использовать их в парных статистических тестах.

В некоторых случаях отдельные показатели могли отсутствовать для одной из страниц. В таких ситуациях соответствующее значение в таблице принимало значение NaN.

Для непараметрических статистических тестов, используемых в данной работе, наличие пропусков не является критичным. Пары наблюдений, содержащие пропущенные значения для конкретной метрики, автоматически исключаются из расчёта статистики теста. Поэтому итоговое число наблюдений для различных метрик может незначительно отличаться.

После фильтрации и подготовки данных общее число корректных пар страниц составило:

N = 121 для анализа структурных метрик (гипотеза H2) и большинства показателей доступности.

Для интегрального показателя доступности AIM-Score итоговое число наблюдений составило N = 119, поскольку в двух корректно загруженных парах отсутствовало значение AIM-Score хотя бы для одной из версий страницы. Поэтому число наблюдений в статистических тестах может отличаться для разных метрик: для большинства структурных показателей использовалось 121 парное наблюдение, а для AIM-Score — 119.

Все этапы обработки данных реализованы в виде автоматизированных скриптов Python, что обеспечивает воспроизводимость анализа и позволяет повторить расчёты для новых наборов данных без изменения методики обработки.

3.3 Формирование аналитических таблиц и метрик исследования

После выполнения этапов автоматизированного сбора данных и формирования агрегированных таблиц выполнялся расчёт показателей, используемых в статистическом анализе. На входе данного этапа используются данные, собранные на предыдущем этапе (раздел 3.2). Скрипт «a11y_build_tables.py» агрегировал данные из JSON-файлов, извлекал DOM, axe-core, WAVE/AIM и «manual»-показатели, рассчитывал производные метрики и формировал таблицу «raw_pages.csv» и парную таблицу «pairs.csv» «стандартная версия / версия для слабовидящих» для последующего статистического анализа.

Данный этап не связан с повторным извлечением информации из веб-страниц, а представляет собой преобразование уже собранных данных в набор метрик, пригодных для сравнительного анализа. Перечень метрик, сохраняемых при обработке данных, приведён в приложении Б.

Формирование показателей структуры страницы

На основе данных о количестве элементов интерфейса формировались агрегированные показатели, характеризующие структурную сложность веб-страницы. Основным показателем выступает «dom_total_visible» — общее число видимых элементов интерфейса страницы.

Данный показатель представляет собой суммарное количество различных типов элементов страницы, зафиксированных на этапе сбора данных. Он используется как интегральная характеристика сложности структуры страницы.

Дополнительно в анализе использовались частные показатели структуры страницы, отражающие количество элементов отдельных типов:

«links_visible» — количество визуально доступных элементов типа «ссылка»;

«buttons_visible» — количество визуально доступных элементов типа «кнопка»;

«forms_visible» — количество визуально доступных элементов типа «форма»;

«images_visible» — количество визуально доступных элементов типа «изображение»;

«headings_visible» — количество визуально доступных заголовков страницы;

«landmarks_visible» — количество визуально доступных ориентировочных областей страницы, таких как шапка, навигация, основное содержимое, подвал и другие структурные блоки.

Использование отдельных показателей позволяет анализировать изменения структуры страницы более детально и выявлять, какие именно элементы интерфейса изменяются в версиях для слабовидящих.

Для анализа доступности страниц использовались агрегированные показатели, сформированные на основе результатов автоматического аудита. Основным показателем количества нарушений является «axe_nodes_count» — общее число обнаруженных нарушений доступности на странице. Для более точного анализа влияния различных типов контента

дополнительно использовался показатель «axe_nodes_noimg» — число нарушений доступности после исключения ошибок, связанных с изображениями. Данный показатель рассчитывался как разность между общим числом нарушений и числом нарушений, связанных с изображениями.

Использование двух вариантов метрики позволяет отдельно оценить влияние изображений на общий уровень доступности веб-страницы.

Для комплексной оценки доступности страниц использовался интегральный показатель: «aim_score» от WAVE. Этот показатель отражает общую оценку доступности страницы и используется как агрегированная характеристика качества веб-интерфейса, он формируется в процессе проверки страницы с помощью инструмента WAVE. Использование интегральной метрики позволяет сопоставлять страницы различных сайтов и выполнять статистический анализ изменений доступности между версиями страниц.

Подготовка данных для сравнительного анализа

Для выполнения статистических тестов данные преобразовывались в формат парных наблюдений. Каждая строка таблицы представляла пару страниц, относящихся к одному и тому же сайту: обычную версию страницы и версию для слабовидящих. Для каждой метрики формировались столбцы вида: «metric_normal», «metric_low».

Такая структура данных позволяет непосредственно вычислять разности между значениями показателей для двух версий страницы и использовать их в парных статистических тестах.

В результате формировался итоговый набор данных, содержащий пары значений всех анализируемых метрик для 121 сайта, для которых были успешно загружены обе версии страницы.

Полученные показатели использовались на следующем этапе исследования для проверки статистических гипотез о различиях между версиями страниц и о взаимосвязи структуры страницы и её доступности.

3.4 Ручная компонентная переоценка выявленных нарушений

Ручная компонентная переоценка использовалась для проверки того, насколько формальная критичность нарушения, определённая axe-core, совпадает с фактическим влиянием ошибки на пользовательский сценарий. В отличие от автоматического отчёта, где нарушение связывается прежде всего с DOM-узлом и правилом проверки, экспертная оценка рассматривала ошибку в контексте интерфейсного компонента.

Единицей ручного анализа выступал не отдельный HTML-элемент, а функциональный компонент страницы: навигационное меню, ссылка, кнопка, изображение-ссылка, форма, блок контактной информации, карточка, модальное окно или иной элемент, через который пользователь получает информацию либо выполняет действие.

Для каждого выбранного нарушения фиксировались: правило axe-core, автоматический уровень влияния, DOM-локатор или фрагмент разметки, тип компонента, пользовательская функция компонента, наличие доступного имени, корректность HTML/ARIA-семантики, влияние на клавиатурную навигацию и возможность выполнения основного сценария пользователем экранного диктора.

Изначально аудитору даются для интерпретации результатоы axe-core. Данный инструмент присваивает нарушениям уровни критичности, например «minor» (незначительный), «moderate» (умеренный), «serious» (серьёзный) или «critical» (критический). Эти уровни отражают формальную тяжесть нарушения в рамках правила проверки, но не фактическую значимость ошибки для конкретного пользовательского сценария. В рамках компонентного подхода автоматическая критичность нарушения рассматривается как предварительная оценка, требующая экспертной интерпретации. Переоценка позволяет уточнить, является ли ошибка блокирующей, существенно затрудняющей взаимодействие, умеренно влияющей на доступность или практически не влияющей на пользовательский опыт.

Названия ошибок для ручной разметки: «critical», «serious», «moderate», «minor», взяты из системы оценок влияния ошибок модулем axe-core, так как их оценка основана на потенциальном влиянии ошибки на опыт пользователя, но нуждается в экспертном контроле. Кроме этого была введена дополнительная категория «ruin» (блокирующий), используемая при ручном аудите. Данная категория соответствует критическим ошибкам доступности, которые фактически блокируют использование интерфейса пользователями экранных дикторов. К таким ошибкам относятся:

- недоступность элементов навигации при управлении с клавиатуры;
- некорректная озвучка элементов интерфейса экранными дикторами;
- бесконечные циклы загрузки контента;
- размещение критически важной информации (например телефонов или времени работы) исключительно в изображениях без альтернативного текстового представления.

Итог ручной переоценки агрегирован в показатели «manual_minor», «manual_moderate», «manual_serious», «manual_critical» и «manual_ruin». Эти показатели использовались для сравнения с автоматической классификацией axe-core и для проверки гипотезы о расхождении формальной и пользовательской критичности нарушений.

Для поддержки ручной компонентной разметки использовался инструмент «fast_manual_components.py». Он позволял аудитору ввести адрес, загрузить соответствующий «axe.json», выделить HTML-фрагмент интерфейсного компонента, отнести его к типу компонента и сопоставить с автоматически найденными нарушениями axe-core. При работе с компонентом

эксперт мог оставить исходную тяжесть ошибки axe-core либо изменить её в поле «manual_impact», если фактическая пользовательская значимость нарушения отличалась от автоматической оценки.

Следует учитывать, что ручная компонентная переоценка выполнялась не для всей совокупности автоматически выявленных нарушений, а для ограниченного подмножества наблюдений, отобранных для экспертной проверки. Такой подход связан с трудоёмкостью ручного анализа, поскольку для каждого нарушения необходимо было сопоставить DOM-фрагмент с фактическим интерфейсным компонентом, определить его пользовательскую функцию и оценить влияние ошибки на сценарий взаимодействия. Поэтому результаты ручной переоценки используются в работе не как полная переклассификация всех автоматических ошибок, а как способ проверки расхождения между формальной критичностью axe-core и фактической пользовательской значимостью нарушений.

В статистической проверке H4, гипотезы о различиях в автоматической и экспертной оценки, использовались только те наблюдения, для которых была доступна ручная компонентная оценка, поэтому объём подвыборки составил n = 19.

3.5 Статистический анализ результатов исследования

В соответствии с целью исследования и задачами были сформулированы следующие исследовательские гипотезы:

H1. Версии для слабовидящих государственных веб-страниц характеризуются меньшей структурной сложностью по сравнению с базовыми версиями, что проявляется в снижении общего количества DOM-элементов, видимых элементов интерфейса и отдельных групп компонентов.

H2. Версии для слабовидящих страниц имеют более высокие формальные показатели доступности по сравнению с обычными версиями: более высокий показатель WAVE/AIM и меньшее количество автоматических нарушений, выявленных axe-core и WAVE.

H3. Количество автоматических нарушений доступности связано со структурной сложностью страницы: чем больше DOM-элементов и интерфейсных компонентов содержит страница, тем выше ожидаемое количество выявленных ошибок axe-core и WAVE.

H4. Автоматическая критичность нарушения, определяемая инструментом axe-core, не всегда совпадает с фактической пользовательской значимостью ошибки, установленной при ручной компонентной переоценке.

H5. Нарушения, связанные с изображениями и визуальными элементами, существенно влияют на формальные показатели доступности, поэтому сравнение страниц только по общему числу ошибок может искажать интерпретацию результатов. Для проверки данной гипотезы показатели axe-core анализируются как с учётом, так и без учёта нарушений, связанных с изображениями.

H6. Массовый аудит доступности реальных государственных веб-сайтов требует контроля технического состояния страницы, поскольку ошибки навигации, блокировки автоматизированных запросов, ограничения выполнения JavaScript и нестабильная загрузка страниц могут влиять на полноту и достоверность собранных данных.

Для проверки сформулированных гипотез использовалось сочетание количественных статистических методов и анализа технических статусов, полученных в процессе массового аудита. Гипотезы, связанные со сравнением базовой и адаптированной для слабовидящих версий страниц, а также с взаимосвязью структурной сложности интерфейса и количеством нарушений доступности, проверялись с использованием непараметрических статистических методов. Такой выбор обусловлен тем, что распределения показателей доступности и структурных характеристик веб-страниц не предполагаются нормальными, а сравнение базовой и адаптированной для слабовидящих версий выполняется для связанных наблюдений, представляющих разные версии одного и того же веб-ресурса.

Для анализа различий между связанными выборками применялся критерий знаковых рангов Уилкоксона. Он использовался при сравнении обычных версий и версий для слабовидящих по количеству DOM-элементов, числу видимых элементов интерфейса, количеству ошибок axe-core, показателям WAVE/AIM и производным метрикам доступности. Для исследования взаимосвязи между структурной сложностью страницы и количеством выявленных нарушений применялся коэффициент ранговой корреляции Спирмена.

Гипотезы, связанные с ручной компонентной переоценкой ошибок, проверялись путём сопоставления автоматической критичности нарушений, определённой axe-core, с экспертной оценкой пользовательской значимости этих нарушений. При этом учитывалось, к какому интерфейсному компоненту относится ошибка, какую пользовательскую функцию выполняет данный компонент и может ли выявленное нарушение препятствовать выполнению основного пользовательского сценария.

Гипотеза о влиянии изображений и визуальных элементов на формальные показатели доступности проверялась путём сравнения метрик axe-core в двух вариантах: с учётом всех выявленных нарушений и после исключения нарушений, связанных с изображениями. Такой подход позволяет отделить общую оценку доступности страницы от эффекта большого количества визуальных элементов, которые могут существенно увеличивать число формальных ошибок, но не всегда одинаково влияют на пользовательский сценарий.

Гипотеза о технической нестабильности массового аудита проверялась на основе анализа статусов и ошибок, зафиксированных в процессе сбора данных. В частности, учитывались ошибки навигации, невозможность получения стабильного DOM, проблемы выполнения JavaScript, ошибки получения снимка страницы, статусы обработки WAVE и случаи, когда страница формально

продолжала загружаться, препятствуя автоматическому сохранению её состояния. Эти данные использовались для оценки ограничений автоматизированного аудита и условий, при которых результаты проверки требуют дополнительной верификации.

Уровень статистической значимости для количественных сравнений был принят равным α = 0,05. При проведении множественных статистических сравнений применялись корректировки значимости с использованием методов Бенджамини–Хохберга и Бонферрони. Статистические результаты интерпретировались совместно с данными ручной компонентной переоценки и техническими статусами сбора, поскольку формальное наличие статистически значимых различий не всегда означает эквивалентное изменение фактической пользовательской доступности интерфейса.

3.5.1 Логика проверки гипотез и используемые статистические методы

Во введении сформулированы шесть исследовательских гипотез. Для устранения логической несогласованности дальнейший анализ сохраняет эту же нумерацию:

H1 — относится к структурной сложности обычных версий и версий для слабовидящих;

H2 — к формальным показателям доступности;

H3 — к связи сложности страницы с числом нарушений;

H4 — к расхождению автоматической и экспертной оценки;

H5 — к влиянию изображений;

H6 — к технической нестабильности массового аудита.

Для парного сравнения обычных версий и версий для слабовидящих использовался критерий знаковых рангов Уилкоксона, поскольку две версии относятся к одному и тому же сайту, а распределения метрик не предполагаются нормальными. Для анализа связи между структурной сложностью и доступностью применялся коэффициент ранговой корреляции Спирмена.

Для каждой пары страниц рассчитывалась разность значений метрики:

$$d_i = x_i^{normal} - x_i^{low}, \quad (2)$$

где:

x_i^{normal} — значение метрики для обычной версии страницы, а x_i^{low} — значение той же метрики для версии для слабовидящих.

Положительное значение d_i означает, что показатель выше в обычные версии, отрицательное — что показатель выше в версии для слабовидящих.

Плотность автоматических нарушений относительно видимой структуры страницы рассчитывалась как:

$$axe_ratio_all = N_{axe} / N_{visible}, \quad (3)$$

где N_{axe} — количество узлов DOM, для которых axe-core выявил нарушения,

а $N_{visible}$ — количество видимых элементов интерфейса.

Для анализа нарушений без учёта изображений использовалась метрика:

$$axe_ratio_noimg = N_{axe_noimg} / N_{visible_noimg}, \quad (4)$$

где N_{axe_noimg} — количество нарушений axe-core после исключения связанных с изображениями и контрастностью ошибок,

а $N_{visible_noimg}$ — количество видимых элементов без учёта изображений.

Для сравнения автоматической и экспертной тяжести нарушений рассчитывалась средневзвешенная оценка:

$$SeverityMean = \frac{1 \cdot minor + 2 \cdot moderate + 3 \cdot serious + 4 \cdot critical + 5 \cdot ruin}{minor + moderate + serious + critical + ruin}, \quad (5)$$

В автоматической оценке axe-core используются уровни «minor», «moderate», «serious» и «critical». В ручной оценке «minor», «moderate», «serious», «critical» и «ruin».

Размер эффекта для связанных выборок рассчитывался как:

$$d_z = \underline{d_i} / s_{d_i}, \quad (6)$$

где $\underline{d_i}$ — среднее значение парных разностей,

а s_{d_i} — стандартное отклонение парных разностей.

Связь между структурной сложностью страницы и показателями доступности оценивалась с помощью коэффициента ранговой корреляции Спирмена:

$$\rho = 1 - (6\Sigma {d_i}^2)/(n(n^2 - 1)), \quad (7)$$

где d_i — разность рангов двух показателей для i-го наблюдения, а n — число наблюдений.

Уровень статистической значимости для количественных сравнений был принят равным $\alpha = 0{,}05$.

Для групп однотипных статистических сравнений дополнительно рассчитывались скорректированные значения p-value с использованием методов Бенджамини—Хохберга и Бонферрони. Корректировка применялась отдельно к группам структурных метрик, формальных показателей доступности и корреляционных проверок. Статистические результаты интерпретировались совместно с размером эффекта, медианой парных разностей, данными ручной компонентной переоценки и техническими статусами сбора, поскольку формальное наличие статистически значимых различий не всегда означает эквивалентное изменение фактической пользовательской доступности интерфейса.

Поправка Бонферрони рассчитывалась как:

$$p_{adj,Bonferroni} = min(p * m, 1), \quad (8)$$

где m — число проверок в соответствующем семействе сравнений.

Для поправки Бенджамини-Хохберга p-value упорядочивались по возрастанию:

$$p_{adj,BH}(i) = min(1, \min_{j \geq i}(\frac{m}{j} p_j)) . \quad (9)$$

Для H6 статистический критерий не применялся: гипотеза проверялась через анализ технических статусов аудита, ошибок загрузки, неполных пар и ограничений выполнения WAVE/axe-core. Поэтому H6 имеет методический, а не корреляционный характер.

3.5.2 Проверка H1: структурная сложность обычных версий и версий для слабовидящих

H1 предполагала, что версии для слабовидящих имеют меньшую структурную сложность. Проверка выполнялась по парным метрикам «dom_total_visible», «links_visible», «buttons_visible», «forms_visible», «images_visible», «headings_visible», «landmarks_visible», поскольку именно видимые элементы формируют пользовательский интерфейс в момент аудита.

Таблица 4 - Проверка H1: структурные различия обычных версий и версий для слабовидящих

Метрика	Статистика	Вывод
видимые элементы DOM	p=0.02036; p_BH=0.055041; p_Bonf=0.142517; Δmed=-9	номинально значимо
ссылки	p=0.024321; p_BH=0.055041; p_Bonf=0.170244; Δmed=0	номинально значимо
кнопки	p=0.987062; p_BH=0.987062; p_Bonf=1; Δmed=0	не значимо
формы	p=0.092654; p_BH=0.129716; p_Bonf=0.64858; Δmed=0	не значимо
изображения	p=3.228e-10; p_BH=2.260e-9; p_Bonf=2.260e-9; Δmed=1	значимо после BH
заголовки	p=0.321181; p_BH=0.374711; p_Bonf=1;Δmed=0	не значимо
ориентиры	p=0.031452; p_BH=0.055041; p_Bonf=0.220162; Δmed=0	номинально значимо

По результатам критерия Уилкоксона наиболее устойчивое различие обнаружено для «images_visible»: n = 121, p = 3,228e-10, медиана разностей = 1, средняя разность = 13,55. Это

показывает, что версии для слабовидящих систематически сокращают или скрывают графический контент.

Для «dom_total_visible» различие также отмечено на уровне alpha = 0,05 (n = 121, p = 0,020360), однако медиана разностей равна -9 при средней разности 77,01. Такое расхождение указывает на неоднородность сайтов: часть страниц действительно упрощается, но это не является универсальным эффектом для всей выборки.

Наиболее вероятное объяснение данного результата связано с тем, что при активации режима для слабовидящих на страницу добавляются элементы панели настройки версии для слабовидящих: кнопки изменения размера шрифта, переключатели цветовой схемы, элементы управления контрастностью, межстрочным интервалом и другие служебные компоненты. Такие элементы увеличивают значение «dom_total_visible», даже если основное содержимое страницы при этом упрощается за счёт скрытия изображений или декоративных блоков.

В рамках настоящего исследования эти элементы не исключались из автоматического подсчёта, поскольку они являются частью пользовательского интерфейса версии для слабовидящих и также должны быть доступны пользователю. Их исключение потребовало бы дополнительной ручной разметки либо разработки алгоритма контекстной идентификации служебных компонентов панели настройки.

Для «links_visible» p = 0,024321, но медиана разностей равна 0, поэтому эффект слабый и требует осторожной интерпретации. Для «buttons_visible», «forms_visible» и «headings_visible» статистически значимого уменьшения не выявлено. Для «landmarks_visible» p = 0,031452, но медиана также равна 0, что указывает на слабый практический эффект.

Результаты показывают, что после поправок на множественные сравнения устойчивым остаётся различие по количеству видимых изображений. Для общего числа видимых элементов, ссылок и структурных ориентиров наблюдаются только номинально значимые различия, а для кнопок, форм и заголовков значимых различий не выявлено. Поэтому H1 подтверждается частично: версии для слабовидящих чаще сокращают визуальный контент, но не демонстрируют системной перестройки всех интерфейсных компонентов.

3.5.3 Проверка H2: формальные показатели доступности обычных версий и версий для слабовидящих

H2 предполагала, что версии для слабовидящих имеют более высокие формальные показатели доступности: более высокий AIM-Score и меньшее количество автоматических нарушений. Для проверки использовались интегральный показатель WAVE/AIM, абсолютное число узлов axe-core с нарушениями, плотность нарушений относительно видимой структуры страницы и показатели без учёта нарушений, связанных с изображениями и контрастностью.

Таблица 5 — Проверка H2: формальные показатели доступности

Метрика	Статистика	Вывод
показатель AIM-Score	версия для слабовидящих > обычная версия; p=1.885e-15; p_BH=9.425e-15; Δmed=-1.2	выше в версии для слабовидящих
количество	обычная версия > версия для слабовидящих; p=8.628e-12; p_BH=2.157e-11; Δmed=8	меньше ошибок в версии для слабовидящих
«ratio_all»	обычная версия > версия для слабовидящих; p=8.713e-7; p_BH=1.452e-6; Δmed=0.013	плотность ниже в версии для слабовидящих
«nodes_noimg»	обычная версия > версия для слабовидящих; p=0.729643; p_BH=0.912054; Δmed=-1	нет устойчивого преимущества
«ratio_noimg»	обычная версия > версия для слабовидящих; p=0.99917; p_BH=0.99917; Δmed=-0.002	преимущество не подтверждено

AIM-Score различие статистически значимо: n = 119, p = 1,885e-15, медиана разностей = -1,20, средняя разность = -1,903. При принятом направлении расчёта это означает, что версии для слабовидящих имеют более высокий интегральный показатель доступности.

Однако для «axe_ratio_all» улучшение не подтверждено: n = 121, p = 0,999999, медиана разностей = 0,01263, средняя разность = 0,022. Для «axe_ratio_noimg_domnoimg» также нет подтверждения устойчивого улучшения: n = 121, p = 0,999170, медиана разностей = -0,00174, средняя разность = -0,009.

Таким образом, H2 подтверждается только в части показателя AIM-Score. Для автоматических нарушений axe-core подтверждение недостаточно: формальная доступность версий для слабовидящих повышается главным образом за счёт изменения визуального состава страницы, а не за счёт устранения ошибок HTML/«ARIA»-семантики и интерактивных компонентов.

3.5.4 Проверка H3: связь структурной сложности страницы с количеством нарушений

H3 предполагала, что более сложные страницы содержат больше автоматических нарушений доступности. Связь проверялась с помощью коэффициента Спирмена отдельно для обычных версий и версий для слабовидящих

Таблица 6 — Проверка H3: корреляции структурной сложности и доступности

Связь	Статистика	Вывод

Обычная версия		
видимые элементы DOM и ошибки axe-core	rho=0.458;p_BH=2.949e-8	больше DOM — больше ошибок
видимые элементы DOM и AIM-Score	rho=-0.374; p_BH=8.571e-6	больше DOM — ниже «AIM»
все элементы DOM и ошибки axe-core	rho=0.483; p_BH=6.129e-9	больше DOM — больше ошибок
все элементы DOM и AIM-Score	rho=-0.386; p_BH=4.625e-6	больше DOM — ниже «AIM»
Версия для слабовидящих		
видимые элементы DOM и ошибки axe-core	rho=0.638; p_BH=1.739e-14	больше DOM — больше ошибок
видимые элементы DOM и AIM-Score	rho=-0.324; p_BH=2.688e-4	больше DOM — ниже «AIM»
все элементы DOM и ошибки axe-core	rho=0.624; p_BH=5.205e-14	больше DOM — больше ошибок
все элементы DOM и AIM-Score	rho=-0.485; p_BH=2.949e-8	больше DOM — ниже «AIM»

Корреляции статистически значимы после поправок на множественные сравнения. Для axe-core связь положительная: более сложная DOM-структура сопровождается большим количеством нарушений. Для AIM-Score связь отрицательная: более сложные страницы имеют тенденцию к более низкому интегральному показателю доступности. H3 подтверждается, но результат интерпретируется как статистическая связь, а не как доказательство причинности.

3.5.5 Проверка H4: автоматическая и экспертная оценка пользовательской значимости ошибок

H4 предполагала, что автоматическая критичность axe-core не всегда совпадает с фактической пользовательской значимостью нарушения. Для проверки сравнивались

агрегированные оценки тяжести автоматического и ручного аудита на подмножестве страниц, где была выполнена экспертная компонентная переоценка.

Для автоматической оценки использовалась шкала «minor» = 1, «moderate» = 2, «serious» = 3, «critical» = 4. Для ручной оценки «minor» = 1, «moderate» = 2, «serious» = 3, «critical» = 4, «ruin» = 5, отражающий ошибки, блокирующие пользовательский сценарий. Для сравнения использовалась формула средневзвешенной тяжести (5).

Таблица 7 — Проверка H4: сравнение axe-core и ручной компонентной оценки

Показатель	Значение
n	19
W	40
p-value	0,012911
Медиана Δ	-0,042459
Средняя Δ	-0,060066
sd	0,093734
dz	-0,640816
Интерпретация	ручная оценка выше автоматической

Результаты: n = 19, W = 40, p-value = 0,012911, медиана разностей = -0,042459, средняя разность = -0,060066, стандартное отклонение = 0,093734, размер эффекта dz = -0,640816. Значение p-value меньше 0,05, поэтому различие между автоматической и экспертной оценкой статистически значимо.

H4 подтверждается: автоматическая критичность должна рассматриваться как предварительная, поскольку она не учитывает роль компонента в пользовательском сценарии. При этом объём ручной выборки невелик (n = 19), поэтому результат следует использовать как аргумент в пользу комбинированного аудита, а не как окончательную модель переоценки всех ошибок.

3.5.6 Проверка H5: влияние изображений и визуальных элементов

H5 предполагала, что нарушения, связанные с изображениями и визуальными элементами, существенно влияют на формальные показатели доступности. Результаты H1 и H2 подтверждают эту гипотезу: именно «images_visible» демонстрирует наиболее выраженное различие между

обычными версиями и версиями для слабовидящих, а после исключения ошибок, связанных с изображениями, различия по метрикам axe-core практически исчезают.

Таблица 8 — Проверка H5: влияние изображений и визуальных элементов

Показатель	Результат	Интерпретация
«images_visible»	обычная версия > версия для слабовидящих, p = 3,228e-10	версия для слабовидящих статистически значимо сокращает изображения
«axe_nodes_count»	обычная версия > версия для слабовидящих, p = 8,628e-12	общее число ошибок меньше в версии для слабовидящих
«axe_nodes_noimg»	обычная версия > версия для слабовидящих, p = 0,729643	без связанных с визуальными элементами и изображениями ошибок преимущество не подтверждается
«axe_ratio_noimg_domnoimg»	обычная версия < версия для слабовидящих, p = 0,000830	плотность оставшихся ошибок может быть выше в версии для слабовидящих
полное удаление изображений	12 из 121 пар	эффект не универсален, но сокращение изображений систематично

Следовательно, сравнение страниц только по общему числу ошибок может искажать интерпретацию. Сайт может получить более высокий интегральный показатель после скрытия изображений, но при этом сохранить те же проблемы семантики, доступных имён, клавиатурной навигации и структуры компонентов. Поэтому в итоговой методике необходимо отдельно анализировать показатели с учётом изображений и без них.

H5 подтверждается: изображения и визуально-контрастные элементы существенно влияют на формальные показатели. При этом корректнее говорить не об универсальном удалении изображений, а о статистически значимом сокращении визуального контента. Рост показателя «AIM» и снижение общего числа нарушений axe-core не должны автоматически интерпретироваться как исправление пользовательских сценариев.

3.5.7 Проверка H6: техническая нестабильность массового аудита

H6 предполагала, что массовый аудит реальных государственных сайтов требует контроля технического состояния страниц. Эта гипотеза подтверждается методическими данными исследования: из исходного набора адресов часть ресурсов не открывалась стабильно, часть пар была исключена из-за ошибок загрузки или отсутствия одной из версий, а итоговый статистический анализ проводился только по 121 корректной паре.

Таблица 9 — Проверка H6: технические статусы и фильтрация выборки

Показатель	Значение	Значение для анализа
«raw_pages.csv»	278 строк	все обработанные страницы
обычные страницы	141	исходные версии сайтов
страницы для слабовидящих	137	адаптированные версии
потенциальные пары	143	исходная база парного сравнения
неполные пары	8	исключены из парного анализа
пары с ошибками загрузки	14	исключены из статистического сравнения
корректные пары	121	основная выборка для H1, H2, H5
ошибки навигации	18 страниц	фиксирует технические сбои сбора
Нет данных WAVE	2 страницы	WAVE/AIM не интерпретируется без повторной проверки

Дополнительно в ходе сбора фиксировались блокировки автоматизированных запросов, промежуточные страницы ожидания, проблемы сертификатов, ограничения выполнения JavaScript, нестабильность динамического контента и необходимость сохранения MHTML-копий после запуска WAVE. Эти факторы могут менять как состав DOM, так и результаты автоматических инструментов.

H6 подтверждается на уровне методики исследования. Контроль «navigation_error», «wave_status», наличия стабильного DOM и корректной MHTML-копии является не вспомогательной технической процедурой, а условием достоверности результатов аудита.

3.5.8 Сводный итог проверки гипотез

H1 — частично подтверждена: версии для слабовидящих упрощаются главным образом за счёт изображений; системного уменьшения всех компонентов нет.

H2 — частично подтверждена: AIM-Score выше, но метрики axe-core без изображений не показывают устойчивого улучшения.

H3 — подтверждена: структурная сложность умеренно связана с числом нарушений и снижением AIM-Score.

H4 — подтверждена: экспертная компонентная оценка отличается от автоматической критичности axe-core.

H5 — подтверждена: изображения существенно влияют на формальные показатели доступности.

H6 — подтверждена: массовый аудит требует контроля технических статусов и состояния страницы.

Итоговая интерпретация показывает, что версии для слабовидящих не должны рассматриваться как полноценная замена доступной реализации основного интерфейса. Они могут улучшать отдельные формальные показатели, но не гарантируют исправления HTML/ARIA-семантики, доступных имён, клавиатурной навигации и пользовательской значимости компонентов.

ГЛАВА 4. РЕКОМЕНДАТЕЛЬНАЯ МОДЕЛЬ АУДИТА И РАЗРАБОТКИ ДОСТУПНЫХ ВЕБ-ИНТЕРФЕЙСОВ

На основе результатов исследования предложена рекомендательная модель аудита и разработки доступных государственных веб-сайтов. Данная модель может быть использована в таких сценариях, как долговременный мониторинг функциональной доступности списка страниц, массового аудита с агрегацией результатов автоматических методов проверки, а также адресной компонентно-ориентированной проверки состояния сайтов. Предоставляемый набор приёмов и рекомендаций нельзя трактовать как единственно верный вариант поиска ошибок и их решений, их задача — создать доступную систему контроля состояния сайтов и агрегации показаний для дальнейшего использования в экспертном контроле и разработке. Предоставить функционал для формирования отчёта имеющего практическую значимость для программистов.

Входными данными модели являются:

- список адресов для проверки;
- готовые связки показателей страниц;
- одиночные адреса;

- файлы компонентного анализа страниц, при наличии исходных данных для его построения.

Так как блоки кода не зависят друг от друга, то при контроле входных данных можно их комбинировать для решения различных задач аудита и контроля, создавать новые программы на основе открытого исходного кода.

Данные формировались модулями «a11y_info_aggregation.py», «a11y_build_tables.py», «metrics_aggregation.py» и «fast_manual_components.py», в них уже интегрирован контроль входных данных.

4.1 Контроль функционирования списка страниц

Для контроля функциональной доступности списка страниц необходимо регулярно активировать скрипт с обходом страниц и их открытием, осуществлять запись о состоянии страниц из списка в одном документа. Решение данной задачи происходило в операционной системе Windows с помощью кода на Python, но также возможен перенос на Linux системы.

Для активации скрипта предлагается использовать bat-файл, текстовый файл с набором команд для выполнения в командной строке ОС Windows, и системное приложение «Планировщик задач». Код для аудита страниц необходимо взять из «a11y_info_aggregation.py» или «bulk_page_open_check.py», блок-схема с задачами кода для аудита приведена на рисунке 4.

Во время открытия страниц можно выделить несколько режимов, влияющих на результаты сбора данных. К ним относятся:

- Отображение браузера при загрузке
- Игнорирование ошибок HTTPS/сертификата
- Введение конкретного имени CSV файла

Загрузка сайта без отображения страниц может привести к блокировке запроса так как некоторые сайты умеют определять автоматизированный/headless - браузер. В автоматическом режиме сайт может:

- не загрузить часть контента;
- показать антибот-защиту;
- зависнуть на проверке;
- отдать другую версию страницы;
- заблокировать переход;
- бесконечно грузить JavaScript.

Кроме этого на страницах существуют «lazy-load» компоненты, они загружаются только когда попадают в окно браузера. К ним могут относиться:

- изображения;

- iframe;
- рекламные блоки;
- виджеты;
- часть JS-компонентов;
- динамические секции.

В режиме «--show-browser» браузер выглядит ближе к обычному пользовательскому Chrome, поэтому такие сайты иногда открываются успешнее, но время проверки увеличится. С учётом этих компонентов в режим проверки с загрузкой страниц лучше внедрить прокручивание страницы до конца, на данный момент осуществляется только факт открытия и стабилизации DOM-структуры.

Также возможно расхождение ошибок, в режиме фоновой загрузки сайт получит статус «stabilization_failed: networkidle timeout», но откроется в видимом и наоборот, тяжёлый сайт может грузится дольше в видимом режиме и чаще получать timeout ошибки.

Так как процессы загрузки страниц имеют как плюсы, так и минусы, лучше их объединить в двухэтапный процесс проверки. Сперва список сайтов обходится с помощью фоновой загрузки, а спорные ресурсы подвергаются повторной проверке с загрузкой в браузере. В таком двухэтапном стеке проверки аудитору необходимо решить задачу передачи спорных ресурсов на второй этап.

Параметр «игнорирование ошибок HTTPS/сертификата» тоже важен в процессе аудита, без него проверка прервётся если у страницы:

- просроченный SSL-сертификат;
- самоподписанный сертификат;
- несовпадение домена и сертификата;
- сайт открывается только по-старому/кривому HTTPS.

В Playwright такая страница может завершиться ошибкой, поэтому явно прописывается режим игнорирования ошибок сертификации, --ignore-suspicious, тогда браузер не считает HTTPS-ошибку причиной немедленного провала.

Ошибки, которые не обходит такой режим:

- Google Safe Browsing пометил сайт как phishing/malware;
- корпоративная политика Chrome запрещает переход;
- браузер блокирует загрузку до создания DOM;
- сайт требует CAPTCHA;
- сайт блокирует автоматизированный браузер;
- сайт зависит от геолокации, cookies, логина или JavaScript-проверки;
- сервер режет запросы от headless-браузера;
- страница бесконечно грузится и не достигает «networkidle», маркер завершения загрузки страницы в библиотеке Playwright.

На основе модификации исходного кода программы для полуавтоматической проверки страниц и функциональных ограничений Chromium/Playwright разработан скрипт «bulk_page_open_check.py», ознакомиться с ним можно в репозитории указанном в приложении К.

Запуск программы через командную строку:

python bulk_page_open_check.py --urls-file urls.txt --show-browser --ignore-suspicious,

bulk_page_open_check.py — имя файла, обязательный компонент;

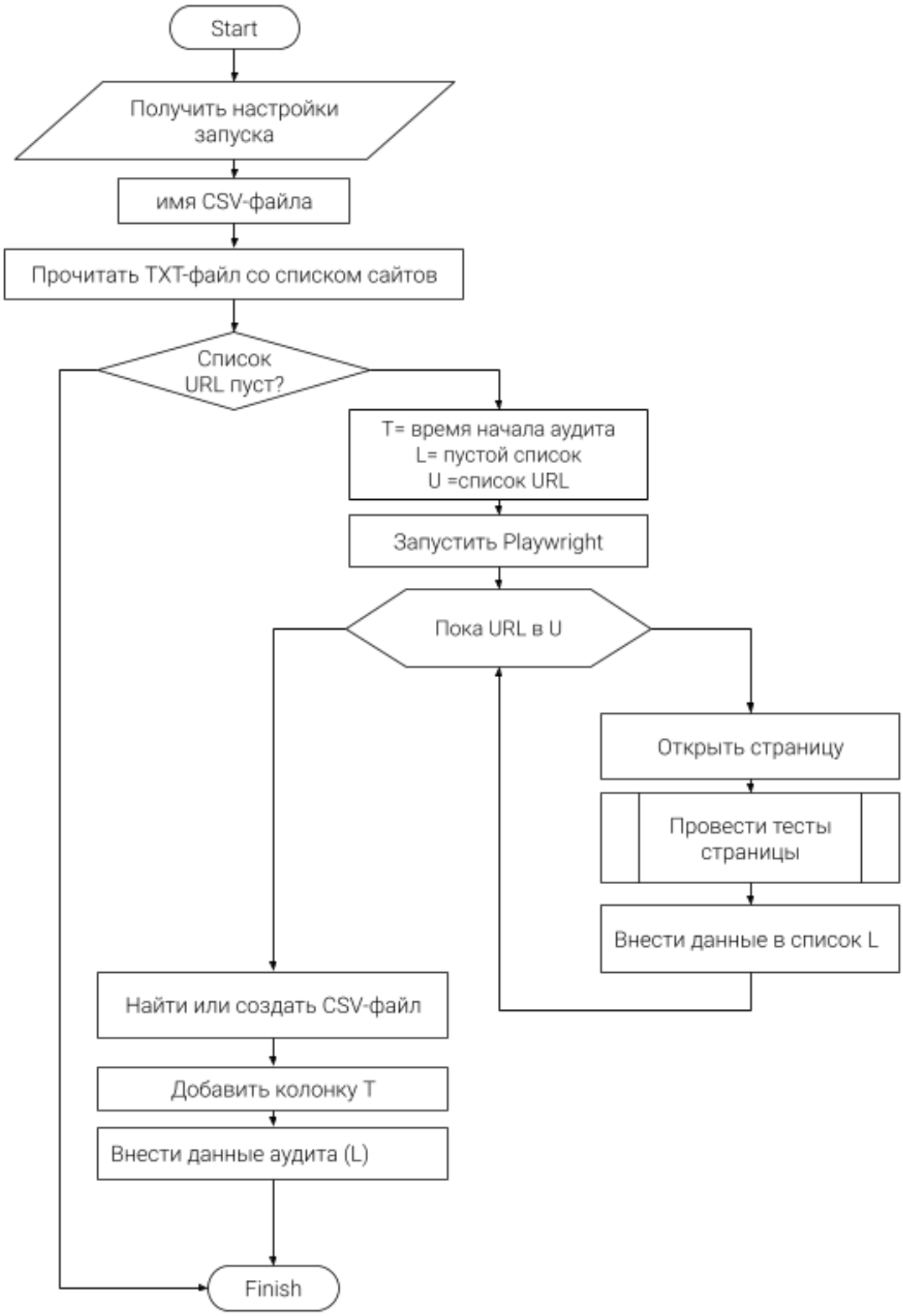
--urls-file urls.txt — источник списка сайтов в файле urls.txt, можно указать путь к файлу, обязательный компонент;

--show-browser — явная загрузка страниц, при удалении страницы будут загружаться в фоновом режиме;

--ignore-suspicious — режим игнорирования ошибок сертификата;

Ознакомиться с всеми вариантами запуска можно в приложении Л.

Рисунок 4 - Блок-схема алгоритма аудита функциональной доступности страниц



4.2 Сбор метрик страниц в автоматическом режиме

Для этого этапа подходят страницы, открывающиеся в видимом режиме и достигающие стабильного состояния, смотрим предыдущий пункт о контроле функционирования страниц, и не требующие дополнительных действий со стороны аудитора. Такие данные можно собирать на основе HTML кода страницы, если он недоступен, нужно извлекать данные из копии страницы, программная структура страницы не искажается при копировании, также на этом этапе возможен сбор результатов axe-core проверки. Если сайт блокирует внешние JS скрипты из-за настроек политики безопасности, то возможно для обхода данного ограничения поможет замена внешнего скрипта на загрузку локальной копии.

Данные оценки WAVE могут быть получены с помощью платного API-сервиса WAVE. В случае возникновений затруднений в использовании такого метода, необходимо или ввести полуавтоматический режим оценки, когда аудитор вручную открывает браузерное расширение WAVE и запускает дальнейшую проверку, такой метод применён для сбора данных в этой работе, или эмулировать клик мышкой с помощью иных программных средств.

На момент проведения аудита взаимодействие Playwright с браузерными расширениями было ограничено моделью безопасности Chromium, предусматривающей изоляцию контекста расширений от автоматизированного управления страницей, вследствие чего запуск и управление расширением WAVE не могли быть надёжно реализованы полностью программными средствами.

4.3 Компонентный режим

Данный режим апробирован при сравнении автоматического определения тяжести ошибок и их ручной интерпретации. Полный цикл работы программы разбит на 3 блока:

1) «capture_engine.py» — автоматически открывает страницу по URL, собирает MHTML-снимок, DOM, результаты axe, runtime-ошибки и создаёт заготовку для WAVE.
2) «bundle_manager.py» — управляет папками результатов, manifest-файлом, проверкой валидности и актуальности собранного bundle.
3) «fast_manual_components.py» — позволяет вручную добавлять HTML-компоненты в JSON, связывать их с axe-нарушениями и продолжать уже начатую работу.

Основной код «fast_manual_components.py» включает в себя предыдущие компоненты. Программа контролирует несколько типов пользовательского ввода:

Во-первых, контролируется выбор основного действия. Пользователь может ввести только 1 или 2: либо продолжить заполнение существующего файла, либо начать исследование нового URL. Все остальные значения отклоняются с сообщением об ошибке и повторным запросом ввода.

Во-вторых, контролируется ввод ответа в цикле добавления компонентов. На вопрос: «Добавить компонент?», — программа принимает только y или n. Если введено другое значение, она не продолжает работу, а просит ввести корректный вариант.

В-третьих, проверяется наличие файлов. При продолжении работы программа ищет существующий JSON-файл по URL. Если файл не найден автоматически, она просит указать путь вручную и повторяет запрос, пока пользователь не введёт путь к реально существующему файлу. Аналогично при загрузке «axe.json»: если файл не найден, программа не падает, а просит указать новый путь.

В-четвёртых, контролируется выбор типа компонента. Тип можно выбрать номером из списка или написать названием. Если номер выходит за пределы списка, ввод отклоняется. Если введённого типа нет, программа показывает доступные типы и предлагает либо выбрать существующий, либо создать новый пользовательский тип.

В-пятых, программа защищает данные от повторного внесения одинаковых компонентов. Для каждого HTML-фрагмента считается хэш после нормализации HTML. Если такой хэш уже есть в индексе, компонент считается дубликатом, программа показывает его существующий тип и id, а повторное добавление не выполняет.

В-шестых, в автоматическом модуле контролируется режим открытия «bundle». Через CLI параметр «--bundle-mode» может принимать только заданные значения: «reuse_if_fresh», использовать, если данные собраны меньше суток назад, «reuse_if_actual», использовать, если данные собраны в одно время, или «always_recollect», всегда собирать набор данных для анализа заново. Это защищает программу от случайного ввода несуществующего режима и обеспечивает согласованность данных при командном аудите так как источники можно сравнить между собой и получить данные о временном или постоянном наличии ошибки.

Отдельно стоит отметить ограничение: поле «mode» в ручном модуле предлагает «normal/low_visual», но жёсткой проверки именно этих двух значений там нет. Если пользователь введёт другое значение, оно будет сохранено как строка режима.

В программе есть два близких, но разных понятия режима.

Первое — режим анализа страницы: «normal» или «low_visual». Он сохраняется в JSON или «manifest» и нужен, чтобы различать проверяемую версию сайта. В ручном модуле этот режим вводится при создании нового аудита.

Второе — режим работы с «bundle», то есть решение: использовать старые данные или собирать страницу заново.

4.4 Преимущества компонентного режима

Не все аудиторы достаточно подготовлены для инспекции кода, также axe-core ошибки представляют из себя набор ошибок с данными о компонентах в виде «xpath», которые нужно интерпретировать в человекочитаемый вариант. Поэтому согласование ошибки и выделенного компонента помогает ориентироваться в полученных данных, выявлять семантические ошибки разметки, которые не всегда очевидны при рассмотрении только автоматического отчёта.

Визуально и функционально элемент может восприниматься пользователем как определённый компонент интерфейса, например кнопка, ссылка, меню, карточка, форма или модальное окно, однако в HTML-разметке он может быть реализован с использованием неподходящих элементов или некорректных ARIA-атрибутов. В таких случаях автоматизированный инструмент фиксирует отдельные нарушения на уровне DOM-узлов, но не всегда связывает их с пользовательской функцией компонента. Выделение компонента непосредственно из кода страницы позволяет увидеть несоответствие между фактической ролью элемента в интерфейсе и его семантическим оформлением: например, использование вместо <button>, отсутствие доступного имени у интерактивного элемента, неправильное применение <role>, некорректное состояние <aria-expanded>, отсутствие связи между <label> и полем ввода или использование <aria-hidden> для значимого содержимого. Поэтому компонентный анализ дополняет автоматическое тестирование не только переоценкой критичности ошибок, но и выявлением случаев, когда сама структура разметки не соответствует пользовательской функции интерфейсного элемента.

Также при анализе аудитов страниц часто фиксировались ошибки, найденные автоматическими инструментами. Но формальные количественные показатели не всегда означают пропорциональное ухудшение пользовательской доступности. Если на странице присутствует большое количество однотипных нарушений, например связанных с изображениями, это может значительно увеличить общее число ошибок. Напротив, небольшое количество ошибок в ключевых интерактивных компонентах может иметь более серьёзные последствия для пользователя, чем большое количество ошибок во второстепенных визуальных элементах. Вручную проверенные сообщения имеют большую ценность при передаче программисту так как в них учитывается контекст использования компонента. Поэтому данный метод создаёт смещение от количественных характеристик к качественному описанию ключевых элементов.

Кроме того, компонентный анализ позволяет Пример ошибок, которые выявлялись при детальном компонентном разборе в таблице 10.

Таблица 10 — Ошибки, зависящие от разметки компонентов

Наблюдаемая ошибка	Пример

Несоответствие html-кода и семантики компонента, отсутствие типа, пример продемонстрирован на рисунке 3.
При таком типе ошибок автоматические способы контроля не могут идентифицировать элемент и корректно его обработать. А приложения для экранного доступа не могут перемещаться между компонентами, например кнопками, и при зачитывании компонента не сообщают о роли.
	Рисунок 3 — Фрагмент списка <button>
	{
	"id": "cmp_0003",
	"html": "<div id=\"wpfront-scroll-top-container\" style=\"opacity: 0.8; right: 20px; bottom: 20px; display: block;\">\n <img src=\"https://xn----8sbflktjqds.xn--p1ai/wp-content/plugins/wpfront-scroll-top/images/icons/36.png\" alt=\"\" style=\"width: auto; height: auto;\"> ",
	"manual": true,
	"axe": {
	"auto_links": [],
	"manual_impact": null
	}
	}

Неверное использование ARIA-role. Кроме некорректно указанного ARIA-role в примере может также встретиться дублирование описаний, <button role="button">.	Фрагмент аудита сайта «БСПБ» (Банк «Санкт-Петербург») «На всех страницах, открываемых из главного экрана (например, «Настройки», «Экран счёта»,), интерфейс воспринимается TalkBack как *диалоговое окно* (role="dialog"). Из-за этого нарушается линейная навигация: пользователь не может перемещаться свайпами между элементами, приходится искать контент касаниями по экрану. Кроме того, элементы не активируются стандартным двойным тапом.>>
Отсутствие alt-текста не всегда фиксируется автоматическими средствами контроля как ошибка. Справа приведён элемент сайта, созданного с помощью конструктора «Tilda». alt-текста индексировалось WAVE как «Feature» (улучшение), а скринридер пропускал его озвучивание.	<img class="tn-atom__img t-img loaded" data-original="https://static.tildacdn.com/tild6633-3732-4539-b761-623661613566/4.jpg" src="4.jpg.webp" **alt=""** imgfield="tn_img_1655392756087">

При выборочном аудите с помощью NVDA у сайта sledcom.ru, рисунок 4, зафиксировали нарушение озвучки стандартной версии. Адаптированная версия, рисунок 5, озвучивалась корректно, но контент был сильно сокращен. Компонентный анализ показал полное соответствие содержимого стандартам оформления, таким образом исключив одну из причин возникновения данной неисправности.

Рисунок 4 - Основная версия сайта sledcom.ru

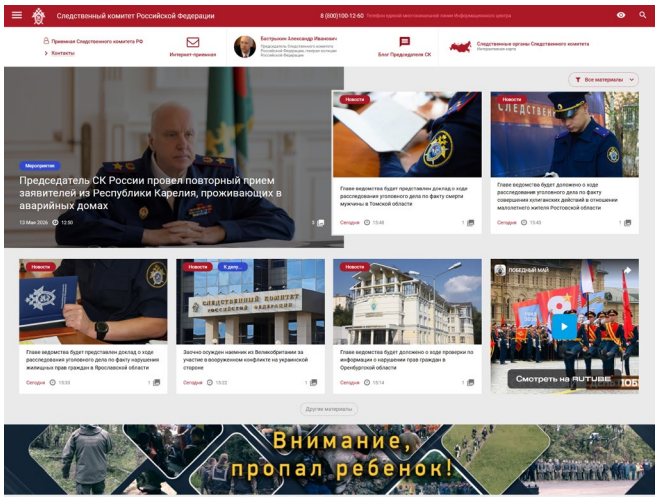
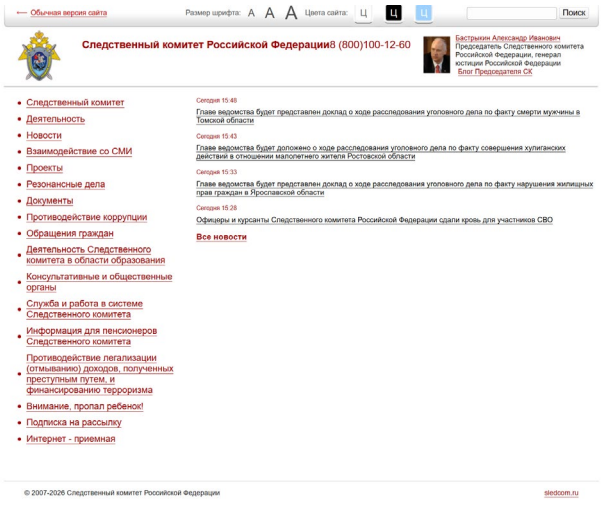


Рисунок 5 - Версия для слабовидящих



4.5 Рекомендации для разработки

Повышение формального AIM-Score не должно быть основной целью разработки. Исправления должны быть направлены на базовую семантику и пользовательские сценарии: корректные HTML-элементы, доступные имена, видимый и программный фокус, связку label-input, текстовые альтернативы значимого визуального контента, предсказуемую навигацию и отсутствие блокирующих динамических компонентов. Версия для слабовидящих может использоваться как дополнительный режим отображения, но не должна заменять доступную реализацию основной версии сайта.

Также в процесс разработки нужно внедрять практики проверки сайтов автоматическими системами аудита, чтобы разработчики, дизайнеры могли оценивать и исправлять макеты и проектную документацию до этапа внедрения продукта. Так как добавленная в последствии версия для слабовидящих не гарантирует доступность государственного сайта. Такие версии могут повышать отдельные формальные показатели, но не заменяют соблюдение WCAG, корректную

HTML/ARIA-семантику, доступные имена интерактивных элементов, клавиатурную навигацию и проверку пользовательских сценариев.

Часть критических ошибок, можно зафиксировать и исправить в ходе разработки при внедрении инструментов экранного доступа в промежуточных тестированиях разрабатываемого веб-контента. При таком подходе не понадобится привлекать команду аудиторов и внедрять исправления в уже выпущенный продукт. В указанном выше примере найденных ошибок компонент соответствует правилам разметки, что входит в стандартный аудит доступности, а выше, что требует дополнительного вовлечения технических специалистов для поиска причин неисправности.

Для лучшего пользовательского опыта также можно провести технический аудит страниц, цель которого выявить устаревшие компоненты и удалить их из системы. Таким образом данные автоматических оценок будут отображать реальное состояние страниц.

ВЫВОДЫ

Проведённое исследование позволило разработать и апробировать компонентно-инструментальный подход к аудиту доступности государственных веб-сайтов. А также выявить существующие ошибки государственных веб-ресурсов, которые пропускаются во время автоматических тестов, на практике. В отличие от проверки, основанной только на автоматическом подсчёте ошибок, предложенный подход объединяет сбор DOM-метрик, результаты axe-core и WAVE/AIM, контроль технического состояния страниц и ручную компонентную переоценку значимости нарушений.

Результаты предложенных методик анализа соответствуют текущим задачам увеличения доступности веб-ресурсов и имеют перспективы для внедрения в процесс разработки и аудита. Снижают требования к необходимому набору знаний о языке гипертекстовой разметки HTML, а также представляют отсутствующую в оцененных аудитах функцию контроля версий сайтов у членов команд. Также сформированные отчёты могут передаваться разработчикам как самостоятельный набор рекомендаций к исправлению с точным указанием задействованных компонентов.

Для проверки поставленных гипотез сформированный набор показателей можно считать исчерпывающим.

Первая гипотеза подтвердилась частично. Версии для слабовидящих действительно отличаются от обычных страниц по ряду структурных характеристик, однако изменения распределены неравномерно. Наиболее выраженное различие связано с сокращением количества видимых изображений, тогда как кнопки, формы, заголовки и часть структурных ориентиров не демонстрируют устойчивого системного уменьшения. Следовательно, адаптированная версия чаще упрощает визуальное представление страницы, чем перестраивает её компонентную и семантическую структуру.

Вторая гипотеза также подтвердилась частично. Интегральный показатель «AIM» от WAVE у версий для слабовидящих выше, однако по метрикам axe-core после нормирования и исключения ошибок, связанных с изображениями, устойчивого улучшения не наблюдается. Это означает, что рост формальной оценки доступности может быть обусловлен удалением или скрытием графического контента, а не устранением базовых ошибок доступности интерфейса.

Третья гипотеза подтвердилась: между структурной сложностью страницы и числом автоматических нарушений выявлена статистически значимая умеренная связь. Более сложные DOM-структуры, как правило, сопровождаются большим количеством нарушений axe-core и более низким показателем AIM-Score. При этом влияние оказывают не только видимые элементы, но и скрытые или технические DOM-узлы.

Четвёртая гипотеза подтвердилась на подмножестве ручной разметки. Автоматическая критичность axe-core не всегда совпадает с фактическим влиянием нарушения на пользовательский сценарий. Ошибка средней формальной тяжести может оказаться блокирующей, если затрагивает ключевую форму, навигацию или кнопку; и наоборот, формально серьёзная ошибка во второстепенном декоративном элементе может иметь меньшую пользовательскую значимость.

Пятая гипотеза подтвердилась. Изображения и визуальные элементы существенно влияют на формальные показатели доступности. Поэтому сравнение страниц только по общему числу ошибок или только по интегральному индексу может приводить к неверному выводу о качестве интерфейса. Для корректной интерпретации необходимо анализировать метрики как с учётом изображений, так и без них.

Шестая гипотеза подтвердилась методически. Массовый аудит реальных государственных веб-сайтов требует контроля состояния страницы: ошибок навигации, блокировок автоматизированных запросов, нестабильной загрузки, ограничений выполнения JavaScript, корректности WAVE-анализа и сохранения MHTML-копии. Без фиксации этих статусов результаты автоматического аудита могут отражать не доступность интерфейса, а техническое состояние загрузки в момент проверки.

Главный вывод работы состоит в том, что доступность государственного сайта должна обеспечиваться прежде всего корректной разработкой страниц и системной фиксацией ошибок, а не последующей адаптацией интерфейса. Автоматизированный аудит должен использоваться как регулярный инструмент выявления нарушений, но его результаты необходимо дополнять ручной компонентной проверкой, указанием технического состояния страницы, затронутого элемента и фактического влияния ошибки на пользовательский сценарий. Отдельного контроля требуют скрытые элементы и устаревший код, поскольку они также учитываются автоматическими средствами аудита и могут искажать оценку доступности. Такой подход позволяет не только оценивать доступность сайта, но и передавать разработчикам воспроизводимые рекомендации для исправления и дальнейшего контроля качества.

ОГРАНИЧЕНИЯ ИССЛЕДОВАНИЯ

Проведённое исследование имеет ряд ограничений, обусловленных как особенностями применяемых методов анализа, так и характеристиками исходной выборки веб-ресурсов.

Одним из ограничений является объём и структура исследуемой выборки. В рамках работы был проанализирован массив из более чем 140 веб-сайтов, для которых удалось сформировать 121 корректную пару страниц, включающую основную версию интерфейса и версию страницы для слабовидящих пользователей. При этом выборка не агрегировалась по уровням государственного управления (федеральному, региональному и муниципальному), а также не включала сайты медицинских учреждений, организаций социальной защиты и иных государственных учреждений. Указанные особенности могут ограничивать возможность экстраполяции полученных результатов на всю совокупность российских веб-ресурсов государственного сектора.

Результаты автоматизированного анализа доступности также зависят от используемых инструментов аудита и параметров программной среды. В рамках исследования применялись системы WAVE и axe-core; анализ выполнялся в браузере Google Chrome for Testing версии 143.0.7499.4 (64 бит) на платформе Windows. Различные инструменты автоматической проверки могут использовать неодинаковые наборы правил, методы обработки DOM-структуры и подходы к интерпретации нарушений стандартов WCAG. Кроме того, на результаты проверки оказывает влияние используемый браузер и его версия, поскольку механизмы обработки данных, поддержка веб-стандартов и особенности работы браузерных движков могут различаться. Это способно приводить к расхождениям как в количестве выявленных ошибок, так и в их классификации по степени критичности.

Дополнительным ограничением является частичный характер проведённого экспертного аудита. Ручная оценка доступности выполнялась не для всей исследуемой выборки, а лишь для части веб-страниц. Кроме того, в рамках исследования не проводилось сопоставление результатов независимыми внешними экспертами, что могло бы обеспечить более высокий уровень согласованности оценок и снизить влияние субъективного фактора. Несмотря на то что выбранный подход позволил выявить качественные различия между автоматизированной и экспертной оценкой доступности, расширение объёма ручной проверки и привлечение дополнительных специалистов могли бы повысить надёжность и статистическую устойчивость полученных выводов.

Следует учитывать и то, что анализ доступности в рамках данной работы проводился преимущественно на уровне HTML-структуры страниц и результатов автоматизированных отчётов инструментов аудита. Полноценное пользовательское тестирование с участием людей с нарушениями зрения в исследование не включалось. Проведение подобных испытаний позволило бы дополнительно оценить влияние выявленных проблем доступности на фактический пользовательский опыт взаимодействия с веб-интерфейсами.

Отдельного внимания требует интерпретация интегральных показателей доступности, в частности AIM-Score. Значение данного показателя может зависеть не только от качества реализации требований WCAG, но и от структурных характеристик веб-страницы, включая объём визуального контента и общую сложность интерфейса. Например, уменьшение количества изображений или декоративных элементов способно приводить к повышению интегральной оценки доступности даже при отсутствии существенных изменений в корректности реализации требований стандартов.

Таким образом, полученные результаты следует интерпретировать с учётом обозначенных методологических и технических ограничений. Несмотря на это, проведённое исследование позволяет выявить общие закономерности в обеспечении веб-доступности и определить характерные проблемы реализации версий сайтов для слабовидящих пользователей. Дальнейшее развитие исследования может быть связано с расширением выборки веб-ресурсов, увеличением объёма экспертной проверки, проведением пользовательского тестирования с участием людей с нарушениями зрения, а также сравнительным анализом доступности в различных программных и аппаратных средах.

НАПРАВЛЕНИЯ ДАЛЬНЕЙШИХ ИССЛЕДОВАНИЙ

Полученные результаты определяют ряд направлений дальнейших исследований в области анализа доступности веб-ресурсов.

Одним из направлений является систематизация способов реализации интерфейсных компонентов с учётом контекста их использования. Анализ структуры веб-страниц позволяет не только выявлять типичные нарушения требований доступности, но и формировать рекомендации по альтернативным вариантам реализации элементов интерфейса, соответствующим стандартам WCAG. Такая систематизация может служить основой для разработки практических рекомендаций по проектированию доступных веб-интерфейсов, а также стать основой для внедрения умных алгоритмов тестирования сайтов, где проверки будут основаны на массиве данных из ручных переоценок и комментариев аудиторов. Также необходимо добавить сценарий для описания сложных ошибок, затрагивающих более одного компонента или относящихся к сценарию взаимодействия.

Важным направлением развития алгоритма тестирования является автоматизированный анализ корректности навигационной структуры сайтов. В рамках подобного подхода для каждой ссылки навигационного меню выполняется переход на целевую страницу, после чего проверяется как техническая корректность перехода, так и соответствие содержания страницы назначению ссылки. Реализация такого анализа позволяет выявлять случаи некорректной маршрутизации, включая переходы на нерелевантные разделы сайта, страницы с ошибками или неработающие элементы навигации.

Дополнительным направлением развития исследования является апробация методов в условиях реального аудита страниц сторонними специалистами. Кроме этого сопоставление результатов ручной проверки, выполненной несколькими специалистами, позволяет оценить степень согласованности экспертных заключений и выявить различия в интерпретации критичности нарушений доступности. Анализ различий между результатами аудита, выполненного различными специалистами, представляет интерес не только с точки зрения повышения надёжности интерпретации экспертной оценки, но и как самостоятельный объект исследования. Подобный подход позволит сформировать обоснованную практикой систему рекомендаций по оценке ошибок, исследовать воспроизводимость процедуры аудита доступности, а также выявить паттерны анализа веб-интерфейсов экспертами.

Наконец, дальнейшее развитие работы может быть связано с совершенствованием разработанного программного инструментария для комплексного аудита веб-ресурсов. Перспективными направлениями являются повышение степени автоматизации этапа сбора данных, снижение роли аудитора на этапе первичного анализа страниц, а также разработка средств

представления результатов проверки в структурированном и человекочитаемом виде, удобном для последующего анализа и исправления выявленных нарушений.

В процессе анализа было отмечено, что страницы, на которых возникают ошибки загрузки или ограничения доступа на стороне сервера, делают невозможным доступ к информации. Если при этом технически возможно реализовать выдачу в качестве результата информативной страницы с информацией об адресе, времени работы, почте и контактном телефоне организации — это необходимо делать, также маркировка ошибки должна иметь понятное пояснение.

ЗАКЛЮЧЕНИЕ

Проведённое исследование позволило реализовать комплексный подход к оценке цифровой доступности веб-ресурсов и сформировать практические рекомендации для разработчиков и аудиторов. Поставленная цель — создать методику комбинированной проверки доступности с опорой на автоматизированные инструменты и экспертную оценку — успешно достигнута.

Итоги по этапам работы:

1. Теоретический этап. В рамках изучения цифровых основ цифровой доступности были систематизированы международные стандарты доступности веб-контента (в т.ч. WCAG), раскрыта роль семантической разметки гипертекстового документа и специальных атрибутов для поддержки вспомогательных технологий, описаны особенности взаимодействия программ экранного доступа.
2. Аналитический этап. Проанализированы методы автоматизированного и ручного аудита веб-страниц, выявлены ключевые ограничения автоматических инструментов — корректность определения ошибок из-за ошибок разметки страницы, несогласованность автоматизированной оценки влияния ошибки на пользователя и контекстно зависимой экспертной интерпретации, влияние на результат оценки устаревших и скрытых фрагментов кода, отсутствие взаимодействия с пользовательскими сценариями, и ручных методов проверки — зависимость поиска ошибок разметки от анализа автоматическими инструментами, отсутствие исходных данных автоматического тестирования и копии страницы по которой проводился аудит, долговременный характер анализа без мониторинга присутствия на странице ранее выявленных ошибок, связанных с динамически меняющемся контентом.
3. Формирование набора данных. По официальным спискам государственных ресурсов регионального и федеративного значения создан репрезентативный набор веб-страниц, включающий базовые и адаптированные для пользователей с нарушениями зрения веб-страниц.
4. Разработка программного комплекса. В ходе работы с веб-страницами создан и внедрён программный комплекс для массовой проверки доступности, который позволяет:
    a. собирать данные о функциональной доступности веб-страниц;
    b. собирать данные о внутренней структуре документа и проверять видимость компонентов для пользователей;
    c. запускать автоматизированные проверки доступности и собирать полученные с их помощью данные;
    d. проводить ручную типизацию компонентов веб-страниц и сопоставлять результаты автоматизированных проверок с выбранными компонентами;

e. проводить ручную переоценку влияния ошибки компонента на пользователя.

5. Определение метрик. Сформирован набор количественных метрик для анализа веб-страниц, учитывающий:
    a. компоненты документа, разделяя их по признаку видимости;
    b. количество и степень серьёзности нарушений;
    c. данные проверок интерфейса с применением средств контекстной визуализации;
    d. долю ошибок относительно сложности страниц;
    e. показатели с учётом и без учёта графических элементов.
6. Сравнительный анализ версий. В результате сравнения адаптированной и базовой версий страниц было установлено что у версий для слабовидящих выше интегральная оценка доступности, но при детальном изучении повышение интегральных показателей доступности связано с уменьшением количества изображений в пользовательском интерфейсе, в то же время семантические ошибки разметки не были устранены, а остальные структурные элементы не подвергались видимому влиянию.
7. Ручная компонентная переоценка. Создана программа для ручной классификации компонентов страниц с автоматической привязкой выявленного компонента к ссылающейся на него автоматически найденной ошибке, также аудитору дана возможность вручную оценить влияние ошибки по предложенной шкале.
8. Сопоставление ошибок критичности. В ходе работы произведён частичный ручной аудит компонентов страниц, выявивший контекстно зависимые изменения оценки влияния ошибки на пользователя у однотипных нарушений. Также на этом этапе были обнаружены ошибки несоответствия разметки компонентов и их семантики, что не может быть обнаружено автоматизированными системами.
9. Анализ ограничений. Выявлены и описаны ограничения массового, автоматизированного аудита, связанные с функциональной доступностью веб-страниц, протоколами, ограничивающими запуск ресурсов анализа, репрезентативностью собранных копий страниц.
10. Разработка рекомендаций. На основе проведённого исследования:
    a. сформулированы практические рекомендации по проведению аудита с учётом выявленных ограничений автоматизированных инструментов и приёмов разработки сайтов;
    b. предлагается инструмент сбора, валидизации, учёта найденных ошибок, сохранения версий веб-страниц для упрощения аудита и повышения ценности аудита для разработчиков с помощью указания на конкретные компоненты страниц;

c. в отдельный протокол вынесен контроль функциональной доступности страниц;
d. предлагаются практические рекомендации для разработчиков для учёта и устранения ошибок доступности на этапе разработки;
e. введены дополнения к страницам ошибок на сайтах для повышения их информационной значимости для посетителей.

Практическая значимость:

Результаты исследования имеют практическую ценность для:

- разработчиков веб-интерфейса — как набор практик и методов контроля для создания доступных цифровых продуктов;
- организаций, ответственных за государственные веб-ресурсы, — для повышения их соответствия стандартам доступности;
- специалистов по аудиту веб-доступности — в качестве методики комплексной оценки.

Перспективным направлением дальнейшего развития работы является:

- совершенствования программного комплекса с учётом процесса и задач аудита и формального описания проблем;
- учёт типов реализаций компонентов, связанных с ними автоматических и вручную установленных ошибок, и обучение программ их распознаванию, предикативной систематизации и анализу на вероятные ошибки;
- разработка набора компонентов для государственных сайтов при использовании которого не будут появляться ошибки доступности.

Таким образом, комплексное решение поставленных задач позволило не только достичь цели исследования, но и сформировать научно обоснованный подход к оценке и обеспечению цифровой доступности веб-контента, готовый к внедрению в реальную практику.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Web Accessibility Initiative. Introduction to Web Accessibility [Электронный ресурс]. – Режим доступа: https://www.w3.org/WAI/fundamentals/accessibility-intro/ свободный. Яз. англ. (дата обращения: 07.03.2026).
2. Microsoft. Designing inclusive software in Windows — Windows apps [Электронный ресурс]: Microsoft Learn. – Режим доступа: https://learn.microsoft.com/en-us/windows/apps/design/accessibility/designing-inclusive-software свободный. Яз. англ. (дата обращения: 05.01.2026).
3. The impact of web accessibility overlays : master's thesis by Daniela Kubesch [Электронный ресурс]. – Режим доступа: https://overlays.dnikub.dev/ свободный. Яз. англ. (дата обращения: 08.08.2025).
4. Accessibility Overlays — Why They Fail [Электронный ресурс]. – Режим доступа: https://overlayfactsheet.com/ свободный. Яз. англ. (дата обращения: 08.08.2025).
5. Axe-core: accessibility engine for automated Web UI testing [Электронный ресурс]. – Режим доступа: https://github.com/dequelabs/axe-core свободный. Яз. англ. (дата обращения: 09.03.2024).
6. WAVE Web Accessibility Evaluation Tools [Электронный ресурс]. – Режим доступа: https://wave.webaim.org/ свободный. Яз. англ. (дата обращения: 12.05.2026).
7. Kuz M., Yaremiy I., Yaremii H., Pikuliak M., Lazarovych I., Kozlenko M., Vekeryk D. Methods for evaluating software accessibility – 2025. – № 3. – P. 163–172. – DOI: 10.15588/1607-3274-2025-3-15.
8. Vigo M., Brown J., Conway V. Benchmarking web accessibility evaluation tools // Proceedings of the 10th International Cross-Disciplinary Conference on Web Accessibility. – 2013. – P. 1–10. – DOI: 10.1145/2461121.2461124.
9. Синицын П. Е. Проверка соответствия стандарту WCAG сайтов российского интернета с помощью юзабилити тестирования интерфейса для слабовидящих пользователей // Программные системы и вычислительные методы. – 2018. – № 1 – С. 95-99.
10. Pudich A. S., Grif M. G., Bakaev M. A. Research of the accessibility of Russian universities websites for persons with visual disabilities // RUDN Journal of Informatization in Education. – 2022. – Vol. 19, №1. – P. 29–44. – DOI: 10.22363/2312-8631-2022-19-1-29-44.
11. Arya Asok A. R., Rekha R. V. Digital inclusion in higher education: A web content accessibility evaluation of best Asian university library websites // The Journal of Academic Librarianship. – 2025. – Vol. 51. – №5 – Article 103120 – DOI: 10.1016/j.acalib.2025.103120.

12. Petrie H., Kheir O. The relationship between accessibility and usability of websites // Proceedings of the SIGCHI Conference on Human Factors in Computing Systems. – 2007. – P. 397–406. – DOI: 10.1145/1240624.1240688.
13. Understanding WCAG 2.2 [Электронный ресурс]. – Режим доступа: https://www.w3.org/WAI/WCAG22/Understanding/ свободный. Яз. англ. (дата обращения: 07.03.2026).
14. ISO/IEC 40500:2012 Web Content Accessibility Guidelines (WCAG) 2.0 [Электронный ресурс]. – Режим доступа: https://www.iso.org/standard/58625.html свободный. Яз. англ. (дата обращения: 07.03.2026).
15. Web Content Accessibility Guidelines (WCAG) 2.2 [Электронный ресурс]. – Режим доступа: https://www.w3.org/TR/WCAG22/ свободный. Яз. англ. (дата обращения: 07.11.2025).
16. W3C Accessibility Guidelines (WCAG) 3.0 [Электронный ресурс]. – Режим доступа: https://www.w3.org/TR/wcag-3.0/ свободный. Яз. англ. (дата обращения: 07.11.2025).
17. Web Accessibility Initiative. ARIA Authoring Practices Guide [Электронный ресурс]. – Режим доступа: https://www.w3.org/WAI/ARIA/apg/ свободный. Яз. англ. (дата обращения: 07.11.2025).
18. Vigo M., Arrue M., Brajnik G., Lomuscio R., Abascal J. Quantitative metrics for measuring web accessibility // Proceedings of the International Cross-Disciplinary Conference on Web Accessibility (W4A). – 2007. – P. 99-107. – DOI: 10.1145/1243441.1243465.
19. Российская Федерация. Законы. О социальной защите инвалидов в Российской Федерации : Федеральный закон №181-ФЗ : принят Государственной Думой 20 июля 1995 г. [Электронный ресурс]. – Режим доступа: http://www.consultant.ru свободный. (дата обращения: 09.10.2025).
20. ГОСТ Р 52872–2019. Интернет-ресурсы и мобильные приложения. Требования доступности для людей с инвалидностью. – Москва : Стандартинформ, 2019. – 23 с.
21. ГОСТ Р ИСО 9241-20–2014. Эргономика взаимодействия человек–система. Часть 20. Руководство по доступности оборудования, программного обеспечения и услуг в области ИКТ. – Москва : Стандартинформ, 2015. – 31 с.
22. ГОСТ Р ИСО 9999–2019. Вспомогательные средства для людей с ограничениями жизнедеятельности. Классификация и терминология. – Москва : Стандартинформ, 2019. – 101 с.
23. Приказ Министерства цифрового развития, связи и массовых коммуникаций РФ от 7 ноября 2023 г. №953 «Об утверждении порядка обеспечения условий доступности для инвалидов по зрению официальных сайтов государственных органов». [Электронный ресурс]. – Режим

доступа: https://minjust.consultant.ru/special/documents/document/51030?items=1&page=1 свободный. (дата обращения: 11.12.2025).

24. Косова Е. А. Аналитический обзор сервисов цифровой доступности на официальных сайтах ведущих университетов мира // Высшее образование в России. – Т. 31. – № 4. – 2022 – С. 148-166. – DOI: 10.31992/0869-3617-2022-31-4-148-166.
25. Шутова А. С. Открытое образование для людей с ограниченными возможностями здоровья: задачи дизайна // Академический вестник УралНИИпроект РААСН. – 2018 – №1 (36) – С. 88-91.
26. Lighthouse overview [Электронный ресурс]. – Режим доступа: https://developer.chrome.com/docs/lighthouse свободный. Яз. англ. (дата обращения: 09.03.2024).
27. Pa11y: automated accessibility testing tool [Электронный ресурс]. – Режим доступа: https://pa11y.org свободный. Яз. англ. (дата обращения: 09.03.2024).
28. Назаров Д. М. Методы оценки юзабилити и доступности веб-сайтов электронной коммерции // Международный журнал прикладных наук и технологий «Integral». – 2022. – №4.

Приложение А — параметры для обеспечения доступности официальных сайтов в информационно-телекоммуникационной сети "Интернет" для инвалидов по зрению

а) информация, размещаемая на официальном сайте, беспрепятственно доступна при помощи клавиатуры, ограничения по времени нажатия клавиш которой отсутствуют, а также при помощи вспомогательных технологий, включая программы экранного доступа, беспрепятственно считывающих структуру и последовательность, в которой представлена такая информация;
б) текстовая информация, размещаемая на официальном сайте, масштабируется не менее чем на 200 процентов от исходного масштаба интернет-страницы без применения вспомогательных технологий, без потери функциональности и без появления горизонтальной полосы прокрутки;
в) документы формата «PDF», а также иные документы, представленные на официальном сайте, доступны для чтения при помощи вспомогательных технологий, включая программы экранного доступа, и размечены в соответствии с положениями национального стандарта Российской Федерации или на официальном сайте представлены альтернативные версии таких документов, доступные для чтения при помощи вспомогательных технологий, включая программы экранного доступа;
г) нетекстовая информация, размещаемая на официальном сайте, представлена в альтернативной версии, доступной для чтения при помощи вспомогательных технологий, включая программы экранного доступа;
д) код, необходимый для подтверждения использования контента человеком, а не электронной вычислительной машиной, в случае его применения на официальном сайте представлен на государственном языке Российской Федерации в доступной для инвалидов по зрению форме;
е) заголовки страниц официального сайта и ссылки, размещаемые на таком официальном сайте, содержат описание;
ж) информация, размещаемая на официальном сайте, соответствует критериям оптимальной контрастности, предусмотренным национальным стандартом Российской Федерации;
з) инструкции, описывающие процесс взаимодействия пользователя официального сайта с информацией, размещаемой на официальном сайте, или элементами интерфейса, не опираются

только на их характеристики, воспринимаемые органами чувств пользователей официального сайта (форму, цвет, размер, визуальное расположение, ориентацию и звук)
и) интерфейс официального сайта для передачи данных о выполнении действия и (или) о запросе на обратную связь и (или) для различения визуальных элементов использует не только визуальные характеристики элемента (цвет, форму, размер, визуальное местоположение, ориентацию), но и текст, доступный для чтения с помощью вспомогательных технологий, включая программы экранного доступа, или звук;
к) автоматически обновляемая информация, размещенная на официальном сайте, контролируется (остановка, скрытие, изменение частоты) в соответствии с критериями, предусмотренными национальным стандартом Российской Федерации;
л) элементы интерфейса официального сайта при перемещении фокуса на них не изменяют контекст;
м) формы, предусматривающие ввод информации, при их наличии на официальном сайте содержат описание в текстовом виде. При ошибочном заполнении формы уведомление пользователя официального сайта об ошибке осуществляется доступным для пользователя официального сайта способом и с описанием ошибки в текстовом виде.

```
row = {
"json_stem": jp.stem,
"md5": md5,
"start_url": start_url,
"final_url": meta.get("final_url"),
"version": version,
"timestamp": meta.get("timestamp"),
"navigation_error": data.get("navigation_error"),
"wave_status": meta.get("wave_status"),
"aim_score": aim_score,
"wave_problem_total": wave_problem_total,
"Wave_Error": wave_error,
"Wave_Contrast_Error": wave_contrast,
"Wave_Alerts": wave_alerts,
"Wave_Aria": wave_aria,
"dom_total": dom_total,
"dom_total_visible": dom_total_visible,
"images": images,
"links": links,
"buttons": buttons,
"forms": forms,
"images_visible": images_visible,
"links_visible": links_visible,
"buttons_visible": buttons_visible,
"forms_visible": forms_visible,
"h1_visible": to_int(dom.get("h1_visible")),
"h2_visible": to_int(dom.get("h2_visible")),
"h3_visible": to_int(dom.get("h3_visible")),
"h4_visible": to_int(dom.get("h4_visible")),
"nav_visible": to_int(dom.get("nav_visible")),
"main_visible": to_int(dom.get("main_visible")),
"header_visible": to_int(dom.get("header_visible")),
"footer_visible": to_int(dom.get("footer_visible")),
```

```
"section_visible": to_int(dom.get("section_visible")),
"article_visible": to_int(dom.get("article_visible")),
"ul_visible": to_int(dom.get("ul_visible")),
"ol_visible": to_int(dom.get("ol_visible")),
"dom_noimg_total": dom_noimg_total,
"dom_noimg_visible": dom_noimg_visible,
"headings_total": headings_total,
"headings_visible": headings_visible,
"landmarks_total": landmarks_total,
"landmarks_visible": landmarks_visible,
"axe_nodes_count": axe_nodes_count,
"axe_nodes_image_related": img_related,
"axe_nodes_noimg": axe_nodes_noimg,
"axe_minor": axe_minor,
"axe_moderate": axe_moderate,
"axe_serious": axe_serious,
"axe_critical": axe_critical,
"manual_present": manual_present,
"manual_minor": manual_minor,
"manual_moderate": manual_moderate,
"manual_serious": manual_serious,
"manual_critical": manual_critical,
"manual_reassessed": manual_reassessed,
"axe_ratio_all": axe_ratio_all, # ratios (existing)
"axe_ratio_noimg": axe_ratio_noimg,
"axe_ratio_noimg_domnoimg": axe_ratio_noimg_domnoimg,
"wave_ratio_all": wave_ratio_all,
"wave_ratio_noimg_domnoimg": wave_ratio_noimg_domnoimg,
# ratios (NEW total)
"axe_ratio_all_total": axe_ratio_all_total,
"axe_ratio_noimg_total": axe_ratio_noimg_total,
"wave_ratio_all_total": wave_ratio_all_total,
"wave_ratio_noimg_total": wave_ratio_noimg_total,
}
```

Приложение В — Метрики для проверки гипотезы H4

Метрика	Описание	Источник
«axe_minor»	Количество нарушений низкой критичности, определённых автоматически	axe-core
«axe_moderate»	Количество нарушений средней критичности	axe-core
«axe_serious»	Количество серьёзных нарушений	axe-core
«axe_critical»	Количество критических нарушений	axe-core
«manual_minor»	Количество нарушений низкой критичности по экспертной оценке	ручной аудит
«manual_moderate»	Количество нарушений средней критичности по экспертной оценке	ручной аудит
«manual_serious»	Количество серьёзных нарушений по экспертной оценке	ручной аудит
«manual_critical»	Количество критических нарушений по экспертной оценке	ручной аудит
«manual_ruin»	Нарушения повышенной критичности «ruin» (блокирующий), выявленные экспертом	ручной аудит

Назначение: проверка H4 — сравнение автоматической критичности axe-core и экспертной компонентной оценки пользовательской значимости нарушений.

Приложение Г — Метрики для проверки гипотезы H1

Метрика	Описание
«dom_total_visible»	Общее количество видимых DOM-элементов на странице
«dom_noimg_visible»	Количество видимых DOM-элементов без учёта изображений
«dom_total»	Общее количество DOM-узлов страницы
«dom_noimg_total»	Общее количество DOM-узлов без изображений
«images_visible»	Количество видимых изображений
«links_visible»	Количество ссылок
«buttons_visible»	Количество кнопок
«forms_visible»	Количество форм
«headings_visible»	Количество заголовков
«landmarks_visible»	Количество семантических областей страницы
«nav_visible»	Количество элементов навигации
«main_visible»	Наличие основного содержимого страницы
«header_visible»	Наличие заголовочной области
«footer_visible»	Наличие нижнего колонтитула

«section_visible»	Количество секций
«article_visible»	Количество элементов «article»

Назначение: проверка H1 — сравнение структурной сложности обычных версий и версий для слабовидящих страниц, включая DOM-элементы, изображения, ссылки, кнопки, формы, заголовки и ориентиры.

Приложение Д — Метрики для проверки гипотез H2 и H5

Метрика	Описание
«aim_score»	Интегральный индекс доступности страницы
«axe_nodes_count»	Общее количество нарушений доступности
«axe_nodes_image_related»	Количество нарушений, связанных с изображениями
«axe_nodes_noimg»	Количество нарушений без учёта изображений
«axe_ratio_all»	Отношение количества ошибок к числу элементов DOM
«axe_ratio_noimg»	Отношение количества ошибок без изображений
«axe_ratio_noimg_domnoimg»	Нормированная метрика ошибок без изображений
«wave_problem_total»	Общее количество проблем доступности по WAVE

Назначение: проверка H2 и H5 — сравнение формальных показателей доступности обычных версий и версий для слабовидящих, а также оценка влияния изображений на метрики axe-core и WAVE/AIM.

Приложение Е — Метрики для проверки гипотезы Н3

Метрика	Описание
«dom_total_visible»	Количество видимых элементов DOM
«dom_noimg_visible»	Количество DOM элементов без изображений
«aim_score»	Интегральный индекс доступности
«axe_nodes_count»	Количество нарушений доступности
«axe_ratio_all»	Относительное количество ошибок
«wave_ratio_all»	Относительное количество ошибок WAVE

Приложение Ж — Статистические методы проверки гипотез H1-H6

Гипотеза	Сравниваемые показатели	Тип данных	Статистический метод	Что проверяется
H1	«dom_total_visible», «dom_noimg_visible», «images_visible», «links_visible», «buttons_visible», «forms_visible», «headings_visible», «landmarks_visible» для обычной версии и версии для слабовидящих	Связанные количественные выборки	Критерий знаковых рангов Уилкоксона	Проверка различий в структурной сложности и составе компонентов
H2	«aim_score», «axe_ratio_all», «axe_ratio_noimg_dom noimg» для обычной версии и версии для слабовидящих	Связанные количественные выборки	Критерий знаковых рангов Уилкоксона	Проверка различий в формальных показателях доступности
H3	«dom_total_visible» vs «axe_nodes_count»; «dom_total_visible» vs «aim_score»; «dom_total» vs «axe_nodes_count»; «dom_total» vs aim_score»	Количественные показатели	Корреляция Спирмена	Проверка связи структурной сложности с числом нарушений и AIM-Score
H4	«axe_minor», «axe_moderate», «axe_serious», «axe_critical» vs «manual_minor», «manual_moderate»,	Связанные порядковые/счётные данные	Критерий знаковых рангов Уилкоксона	Проверка расхождения автоматической и экспертной оценки критичности

	«manual_serious», «manual_critical», «manual_ruin»			
H5	«axe_nodes_count», «axe_nodes_image_related», «axe_nodes_noimg», «axe_ratio_noimg_domnoimg», «images_visible»	Связанные количественные выборки	Критерий Уилкоксона + сравнительная интерпретация метрик	Проверка влияния изображений и визуальных элементов на формальные показатели
H6	«navigation_error», «wave_status», наличие MHTML-копии, технические ошибки загрузки и неполные пары	Служебные статусы аудита	Качественный анализ технических статусов	Проверка необходимости контроля состояния страницы при массовом аудите

Приложение И — Формулы используемых метрик

Метрика	Формула / способ расчёта	Обозначения
«axe_nodes_count»	Суммарное число нарушений, выявленных axe-core	Все ошибки axe-core на странице
«axe_nodes_image_related»	Суммарное число нарушений axe-core, связанных с изображениями	Ошибки типа missing «alt», image contrast и др.
«axe_nodes_noimg»	«axe_nodes_count» - «axe_nodes_image_related»	Ошибки без учёта нарушений, связанных с изображениями
«axe_ratio_all»	«axe_nodes_count» / dom_total_visible»	Отношение общего числа ошибок к числу видимых DOM-элементов
«axe_ratio_noimg»	«axe_nodes_noimg» / «dom_total_visible»	Отношение числа ошибок без изображений к числу видимых DOM-элементов
«axe_ratio_noimg_domnoimg»	«axe_nodes_noimg» / «dom_noimg_visible	Нормированное число ошибок без изображений по DOM без изображений
«wave_problem_total»	Общее количество проблем, выявленных WAVE	Сумма ошибок, предупреждений и связанных проблем по WAVE
«wave_ratio_all»	«wave_problem_total» / «dom_total_visible»	Отношение числа проблем WAVE к числу видимых DOM-элементов
«wave_ratio_noimg_domnoimg»	«wave_problem_total_noimg» / «dom_noimg_visible»	Нормированное число проблем без изображений
«dom_total_visible»	Подсчёт всех видимых DOM-элементов страницы	Все отображаемые элементы
«dom_noimg_visible»	Подсчёт всех видимых DOM-элементов без элементов изображений	DOM без <img> и связанных графических объектов

Приложение К —Репозиторий с указанными в работе скриптами

https://github.com/Ockimura/Project_audi

Приложение Л —Режимы запуска bulk_page_open_check.py

Режим / задача	Команда	Что делает
Базовый запуск	python bulk_page_open_check.py --urls-file urls.txt	Проверяет URL из urls.txt, браузер запускается без окна, результат пишется в a11y_page_errors.csv.
Видимый браузер	python bulk_page_open_check.py --urls-file urls.txt --show-browser	Показывает окно Chromium. Удобно для визуальной отладки.
Игнорировать подозрительные страницы	python bulk_page_open_check.py --urls-file urls.txt --ignore-suspicious	Ослабляет HTTPS/security-ограничения, пытается пройти interstitial-страницы Chromium.
Видимый браузер + ignore suspicious	python bulk_page_open_check.py --urls-file urls.txt --show-browser --ignore-suspicious	Лучший режим для ручной диагностики проблемных сайтов.
Свой CSV-файл	python bulk_page_open_check.py --urls-file urls.txt --csv report.csv	Записывает результат в указанный CSV. Если расширения нет, скрипт сам добавит .csv.
Увеличить timeout навигации	python bulk_page_open_check.py --urls-file urls.txt --timeout 60000	Даёт странице больше времени на открытие. Значение указывается в миллисекундах. По умолчанию 45000.
Увеличить число попыток навигации	python bulk_page_open_check.py --urls-file urls.txt --retries 5	Повторяет открытие страницы большее число раз. По умолчанию 3.
Больше попыток стабилизации DOM	python bulk_page_open_check.py --urls-file urls.txt --stable-retries 7	Увеличивает число попыток дождаться стабильной загрузки DOM. По умолчанию 5.
Увеличить timeout стабилизации	python bulk_page_open_check.py --urls-file urls.txt --stable-timeout 12000	Увеличивает ожидание domcontentloaded/networkidle.

		Значение в миллисекундах. По умолчанию 8000.
Изменить задержку перед чтением HTML	python bulk_page_open_check.py --urls-file urls.txt --stable-delay 1.0	Задаёт паузу перед page.content(). Значение в секундах. По умолчанию 0.6.
Каждый URL в новой вкладке	python bulk_page_open_check.py --urls-file urls.txt --new-page-per-url	Открывает каждый URL в свежей вкладке, а не переиспользует одну страницу.
Вернуть exit code 1 при ошибках	python bulk_page_open_check.py --urls-file urls.txt --fail-on-error	Если хотя бы один URL не открылся или DOM не стабилизировался, программа завершится с кодом 1.
Медленные сайты	python bulk_page_open_check.py --urls-file urls.txt --timeout 60000 --stable-timeout 15000 --stable-retries 7	Практичный режим для сайтов, которые долго грузятся.
Максимальная диагностика проблемных сайтов	python bulk_page_open_check.py --urls-file urls.txt --show-browser --ignore-suspicious --new-page-per-url	Видимый браузер, ослабленные ограничения и новая вкладка для каждого URL.